NTU–TUM · NM6008 DIGITAL IC DESIGN

前端到后端
错误诊断与修复手册

Lab 3–5：RTL · VCS · Design Compiler · SDF · SAIF · Power Compiler

Lab 6：Innovus · CTS · Route · DRC · Quantus · GDS

加录：Secret Code / Serial Adder 考试模拟故障链

图文证据 / 原始错误关键词 / 决策分支 / 修复命令 / 验收条件

版本：2026-10-28（夜间整合） · 中文可检索 PDF

来源：历史对话可恢复片段、实验手册、SOP、实验报告及原始图证

目录 · Ctrl+F 搜索优先

00 30 秒查错入口 + 原始错误码索引 3

01 专题 A｜前端：工具 / RTL / TB / VCS 4

02 专题 A｜GF22 综合 / Tcl / Design Vision / STA 11

03 专题 A｜门级 SDF / Power / SAIF 14

04 专题 B｜Innovus 导入 / PG / Floorplan / CTS / DRC 18

05 专题 B｜RC/Quantus 原始错误码 / 豁免边界 25

06 Mock1 / Mock2 实测错误链 + 签核清单 28

07 源文件索引 / 真实性审计 32

搜索建议：优先搜索 UID-3、PWR-362、LBR-1、SDFCOM_CFTC、GLIBC_2.27、NREX-94 等原文。每个错误的标签写明“实遇 / 手册 / 预防”和是否已经取得修复后的证据。

NM6008 Lab 3–6：前端到后端全链路故障诊断图文手册

> **修订**：2026-10-28 夜间 · 图文增强版 · 适用环境：NTU NM6008、Linux/CentOS、Synopsys VCS / Design Vision / Power Compiler、Cadence Innovus、GF22。
> **目标**：出现问题时按症状查编号，执行「定位 → 最小修复 → 验收」，而不是重复点击、盲改工艺库或重跑全部流程。
> **证据分级**：［实遇］我们先前讨论、Mock1 增补 SOP 或最终实验报告明确记录；［手册］2026 版 Lab 3–6 指明；［预防］通用且合理的潜在故障，不声称曾真实发生；［未核实］有现象或待查项，但无法从记录证明修复已经完成。
> **覆盖边界**：对可检索的近期 NM6008 对话片段、Lab 手册、Mock1 增补 SOP、M4/M5/M6 最终报告、M4 原始证据包和 M6 历史操作笔记交叉整理。历史聊天中的部分内嵌图片无法作为独立原始文件批量导出，所以不能证明逐条穷尽所有图片或全部聊天。下文视觉证据分【用户原始实验图】【官方手册操作示例】【基于已记录数值重绘】，未恢复的错误截图用可搜索原文代替。**没有逐条原始证据的错误不冒充「已验证修复」**。命令中 `TOP`、`TB`、`dut`、文件名、路径、时钟约束必须与题目实际设计匹配。

---

0. 考场 30 秒查错入口

先确定**错误出现在哪个阶段**，不要上来重装/删文件。

阶段	关键输入	正确产物/验收	转到
T1 Behavioral RTL	RTL、TB、vector	编译成功、TB PASS、波形正确	§2、§3
T2 Synthesis	RTL、Tcl、GF22 `.db`、constraints	当前 Top 的 GF22 netlist、SDF、报告、可选 SDC/DDC	§4、§5
T3 Gate-level + SDF	综合网表、SDF、TB、`library.f`	SDF 注释确实成功、门级 checks PASS、`inter.vpd`	§6
T4 Power	Gate VPD、`.vcd`、`.saif`、网表、GF22 `.db`	活动注释有效、Power 报告可读	§7
T5 Physical (Lab 6)	`Syn_MatrixMultiplier_IO`、`.globals`、LEF、SDC、libraries	Innovus 可复现布局、routing、时序、DRC 和导出文件	§8

在 Linux 终端先运行的只读排查命令：

```
pwd
ls -lah
find . -maxdepth 4 -type f | sort | head -100
grep -RniE 'Error:|ERROR|FATAL|Warning:|VIOLATED|unresolved|PWR-|SDFCOM|PASS|FAIL' logs ./*.log 2>/dev/null |
head -100
find . -type f \( -name '*.v' -o -name '*.sv' -o -name '*.sdf' -o -name '*.sdc' -o -name '*.saif' -o -name
'*.vpd' -o -name '*.rpt' \) -printf '%TY-%Tm-%Td %TH:%TM %p\n' | sort -r | head -70
```

说明：最后一条为 GNU `find`（CentOS 可用）；`grep` 搜不到不代表工具没报错，先确认真实 log 路径。**旧日志和旧产物时间戳不能证明本次运行成功**。

核心原则：只改最小必要部分；T1 改 RTL 后必须重做 T1→T2→T3→T4，T2 改综合后必须重做 T2→T3→T4，T3 改激励后必须重做 T3→T4。T5 改布局后重新检查相关 routing/STA/DRC/export。保留旧证据/检查点，不要直接删除唯一可用结果。

0A. 本 PDF 的证据可信度与用法（先读）

- **A 类 • 原始证据**：可恢复的实验截图、截图可核对的具体工具输出、最终报告中的具体数据。
- **B 类 • 当时的对话诊断**：能恢复确切错误码、指导修复，但没有后续 PASS/日志；记为“诊断明确，修复待复测”。
- **C 类 • 教学文档/通用排障**：老师手册、考试 SOP 和合理推断。不能假装我们当时曾遇到。
- **不同阶段的错误不能互相套命令**：VCS 找不到 GF22 cell Verilog ≠ DC 的 `.db/link_library` 找不到；SAIF 文件不存在 ≠ 有文件但 hierarchy 映射 0%。
- **考试最短路径**：`Ctrl+F` 搜索真实错误码（如 `UID-3` / `PWR-362` / `IMPEXT-7050`）→ 阅读“判断分支” → 先做只读检查 → 最小修复 → 检验本轮 PASS。

可搜索错误码一览（原始码优先）

工具	Ctrl+F 搜索词	主因优先级	条目
Shell / VCS	`Command not found`, `-sverilog`, `Illegal timescale`, `VCS_SDB_ERROR`	shell、SV 解析、时间单位、旧库	E01/E04/E05/E41
DC / Design Vision	`UID-4`, `UID-3`, `LBR-1`, `LINK-5`, `script_path`, `No clocks`	cwd、Top、目标工艺库与 link	E09/E11/E15/E18/E42
Gate + SDF	`SDFCOM_CFTC`, `WSUM`, `SDF instance`, `library.f`	std-cell 模型、SDF hierarchy	E19-E24
Power Compiler	`PWR-201`, `PWR-362`, `PWR-452`, `0%`	文件链/库/实例层次/映射	E25-E30/E43
Innovus import/PG	`AES_TOP_DP.view`, `MET1(1)`, `MET4(4)`	旧 globals / 老 layer 名	E32/E44
Innovus CTS / DRC	`WNS`, `M1 short`, `verify_drc`, `ecoRoute`, `Killed`	hold、pin/wire 冲突、资源	E34/E35/E46
RC / Quantus	`GLIBC_2.27`, `IMPDC-495`, `IMPEXT-7050`, `NREX-94`, `IMPEXT-2900`	系统版本/提取档位/RC corner	E36/E47-E51

专题 A｜前端：Lab 3 / Lab 4 / Lab 5（含 Secret Code、Serial Adder 模拟考）

1. 先认清 4 种解释器：很多“软件报错”其实只是位置错误

看到的提示符	解释器	可以执行	不要执行
`[user@host ...] $` / `%`	Linux bash / csh	`pwd`、`ls`、`cd`、`vcs`、`./simv`	`read_verilog`、`report_power`
`design_vision>`	Synopsys/DC Tcl	`read_verilog`、`current_design`、`source scripts/*.tcl`	`source xxx_cshrc`
`dc_shell>`	Power Compiler/ DC Tcl	`read_saif`、`report_saif`、`report_power`	shell 的 `vpd2vcd`
`innovus>`	Cadence Innovus Tcl	`init_design`、`placeDesign`、`timeDesign`、`verify_drc`	用 VCS 命令编译 Verilog

典型 `[实遇]`：在 csh 中执行 `source common_setup_gf22.tcl`，出现 `puts: Command not found` 等错误。 Tcl 不能直接拿给 csh 解释；只需在 Design Vision / DC 会话中执行它，原配置文件不是因此坏了。

Lab 3–5 的三种软件环境（手册对应 M3 P8 / P14 / P18–20）

```
# T1: RTL behavior (在当前 Behavioral 目录)
csh
source synopsys_linux_vP_2019_cshrc
which vcs

# T2: 另开一个终端，先 cd 到 Synthesis
csh
source synopsys_linux_vR_2020_cshrc
which design_vision
design_vision &

# T3: 另开终端，cd 到实际存在的 rtl（不一定叫 RTL）
csh
source synopsys_linux_vP_2019_cshrc
which vcs

# T4: 在 rtl/PowerAnalysis，改用 R-2020 环境
csh
source ../../Synthesis/synopsys_linux_vR_2020_cshrc
which dc_shell-xg-t
```

> 路径示例基于 Module3 原始结构；其他 Module 与 Mock 题先执行 `pwd; ls`，不要无脑粘贴。`which` 查不到工具时优先检查是否加载环境，不能直接断言机器没有安装或 License 失效。

---

2. T1 • RTL / Testbench / VCS / DVE 故障

E01 `[实遇/预防]` `vcs: Command not found` / 找不到 `dve`

- **定位**：当前终端未加载 `vP_2019`、路径改了或当前 shell 不对。
- **修复**：回到真实 Behavioral/rtl 目录：`csh` → `source synopsys_linux_vP_2019_cshrc` → `which vcs`；若仍失败，`ls -l` 查看脚本，`echo $PATH` 确认是否新增目录。不要先猜 License。
- **验收**：`which vcs` 显示路径、`vcs` 可启动。若**启动后**报 license，才根据实际许可证诊断。

E02 `[实遇]` `Unknown type 'wire'` / 端口声明 syntax error

- **定位**：Verilog 拼写、逗号/分号、`input wire` 声明、Unicode 全角符号或粘贴污染。
- **修复**：根据 VCS 第一条错误的文件名和行号检查前后 5–10 行；`nl -ba file.v | sed -n '20,55p'`。注意 `wire` 本身不是 SystemVerilog 才有的类型。
- **验收**：当前源码重新编译，0 fatal/syntax errors。不要通过删除报错端口来“修复”。

E03 `[实遇]` `syntax error near always` / `Syntax error token ';'`

- **定位**：`always` 错拼、上方缺 `end` / `endcase` / `;`，`assign` 括号不平衡。
- **修复**：先找上一个语句结束点、`begin/end`、`case/endcase` 成对关系；检查 `&` vs `&&`、`|` vs `||`；重新编译并查看**第一条**错误。
- **验收**：编译成功且 TB 有自动检查通过，不能仅以“有波形”作为功能验收。

E04 `[实遇]` `Illegal timescale for module`

- **定位**：不同输入源的 `timescale` / 编译单位时序设置冲突，或 TB 中 `timescale` 放错位置。
- **修复**：检查 TB 顶部是否为 Verilog 编译指令：

  ```
  `timescale 1ns/1ps
  module tb_example;
  ```

  检查 RTL/综合网表与门库的 timescale 兼容性和 VCS 编译开关。必要时统一**本次编译**的时间尺度，优先沿用 M3 P17 所示 TB 格式。
- **验收**：编译告警消除或得到明确解释，SDF time unit 可正常解释。**禁止为修这个问题去改老师的 GF22 标准单元库**。

```
52      B = 0;
53      $vcdpluson(U1);
54
55      $fdisplay(file1,"No\tB\tA\tCIN\tS\tCOUT");
56
57      for (i = 0; i < N; i = i + 1)
58        begin
59          {B, A, CIN}  = REG_DATA[i];
60
61          #(DELAY);
62          $fdisplay(file1,"%d\t%d\t%d\t%d\t%d\t%d", i, CIN, A, B, S, COUT);
63
64        end
65
66      $fclose(file1);
67      $vcdplusoff(U1);
68      $finish;
```

官方 M3 手册：门级 TB / SDF 设置页面中的原图（教学操作示例；非本轮报错原截图）

E05 `[实遇]` `VCS_SDB_ERROR` / `VFS_SDB_ERROR: generated by different version`

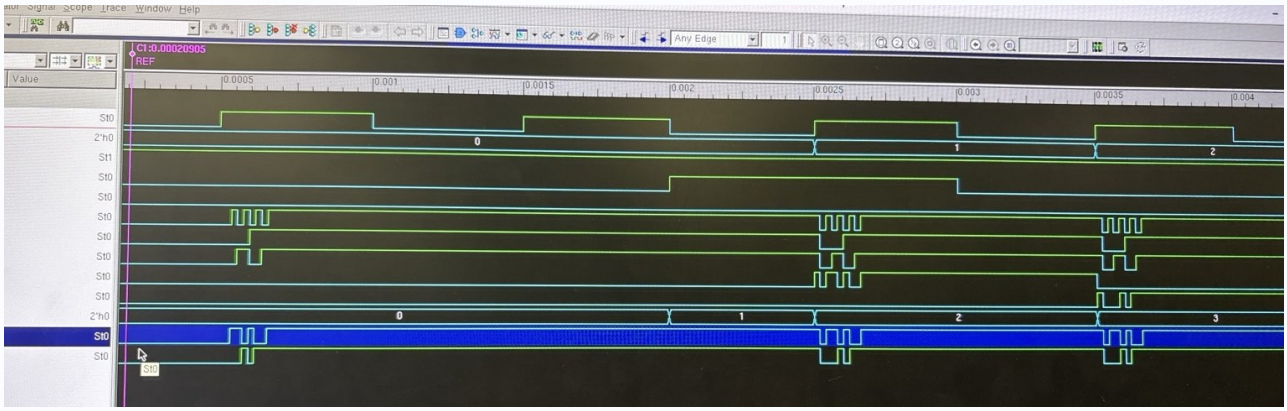
- **定位**：旧 `simv.daidir`/增量数据库是另一个 VCS 版本生成的。
- **修复**：把新的仿真构建放到独立目录；或先备份旧库、用新的输出名与 `-Mdir` 重新编译。例如：

```
mkdir -p build_clean
vcs -debug +v2k ./tb.v ./dut.v -o ./build_clean/simv -Mdir=./build_clean/csrc -l build_clean/compile.log
./build_clean/simv -l build_clean/run.log
```

  上述 `tb.v/dut.v` 只是占位符，**门级仿真还须补 GF22 `-f library.f` 与真实设计文件**。
- **验收**：当前版本构建成功，不再重用旧 `.daidir`；不需要删 PDK。

E06 `[实遇]` DVE 没有波形、全 0、全 X 或“一条直线”

- **定位顺序**：① 仿真是否真的 Run；② 时钟和 NRST 是否有效；③ 波形时间轴是否过宽；④ 信号是否 Add to Waves；⑤ DUT 是否为正确实例；⑥ 是否打开了旧的 VPD。
- **修复**：DVE 选 TB 信号 → Add To Waves → Start/Continue → Zoom Fit 后缩放到 ns 级；检查 `CLK` 翻转、`NRST` 释放、激励、结束时间。
- **验收**：看到与 TB 时间顺序一致的输入/输出变化，并有 checker PASS。X 不是一律错：复位之前的未知值可能正常，但复位后持续 X 必须排查。



用户 M4 证据包：行为级 DVE 波形，便于比对 Reset、状态与采样时间

E07 [实遇/预防] simv 找不到 / GUI 不启动

```
pwd
ls -lh simv simv.daidir 2>/dev/null
./simv -gui
```

若 `./simv` 不存在，说明编译未完成或用了 `-o` 生成别名；先检查 compile log。`simv -gui` 只有当前目录在 PATH 中才可靠，通常应使用 `./simv -gui`。

E08 [预防] TB “PASS”但其实没有有效检查

- 自检应包含 expected/reference、每笔断言或比较、`$fatal`/错误累计、明确 PASS 条件和超时保护。
- 对时序题要驱动与采样错开（例如在 `negedge CLK` 驱动、在适当 `posedge` 后采样）；非阻塞赋值更新发生在 NBA 区，不能同一时间槽立刻比较旧值。
- 对有 SDF 的门级 TB 需留出规范允许的稳定/采样窗口，不能为了消除 FAIL 随意将延迟调得巨大。

---

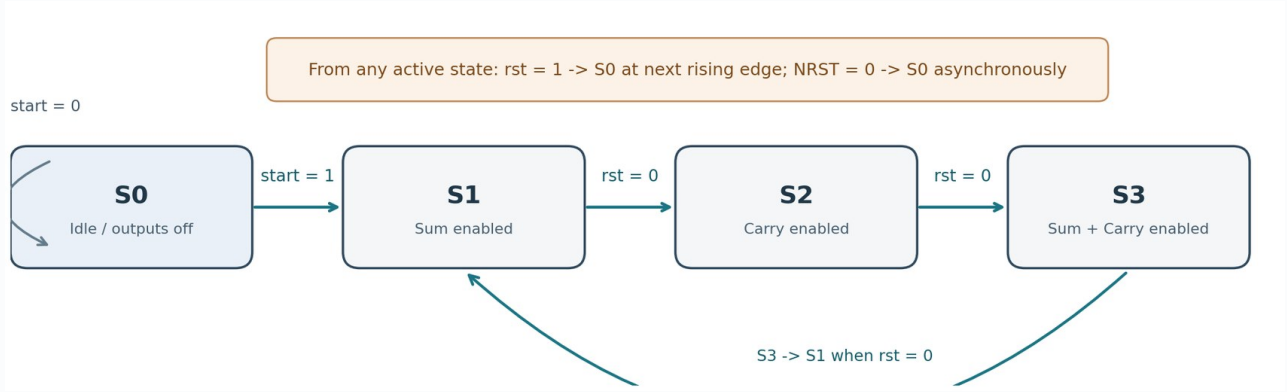
3. 专题：Lab 3 / Lab 4 / Lab 5 的功能性坑

F01 [手册] Lab 3 FullAdder 的三道常见混淆

1. **FullAdder 是纯组合电路**：`S=A^B^CIN`，`COUT=(A&B)|(CIN&(A^B))`，验证 8 组输入组合；不要凭空加 CLK。
2. **Design Vision `Schematic View` 是综合层次/标准单元连接，不是 CMOS 晶体管级图**。GF22 `.db` 含 timing/area/power，不含可供 DC 展开的 PMOS/NMOS schematic。
3. **门数/面积/传播延迟来源不同**：门数查 `report_reference`/`report_cell`，面积查 `report_area`，受约束路径的时序查 `report_timing`，具体输入跳变的输出延迟可从 SDF 门级波形测量。`slack` **不是**单个 gate propagation delay。

F02 [实遇] Lab 4 FSM 异步复位、同步复位、状态掩码搞混

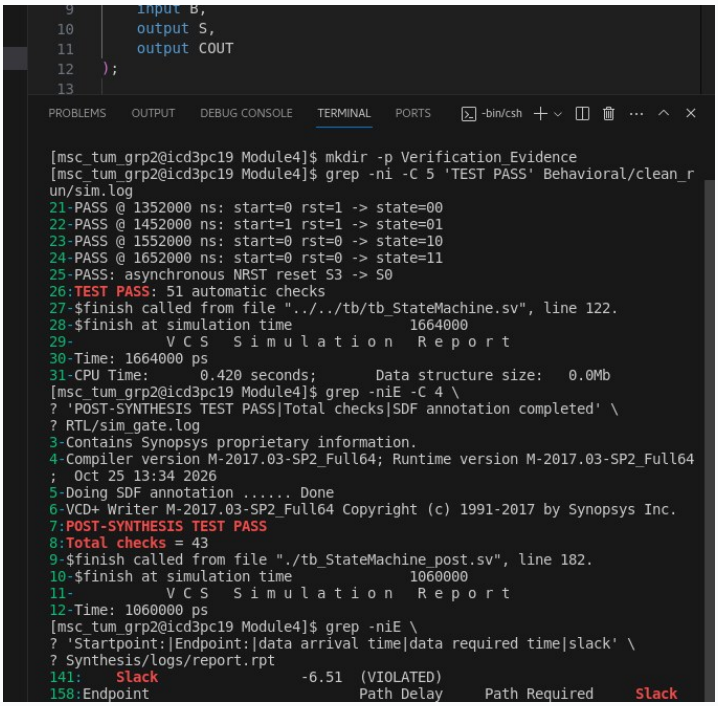
- `NRST=0`：`always @(posedge CLK or negedge NRST)` 使状态异步回 `S0`。
- `rst=1`：在活动状态的**下一个 CLK 正沿**回 `S0`，不要当成异步 `NRST`。
- `S0` 等待 `start`；`S1→S2→S3→S1`。`S1/S3` 开 Sum，`S2/S3` 开 Carry；不输出并不代表没有计算组合全加器逻辑。
- M4 时钟 **period = 100 ns**。TB `forever #50 CLK=~CLK` 的 50 ns 是**半周期**。综合脚本必须真正执行 `create_clock -period 100 ...`，只设置变量不等于生成时钟。
- **验收**：覆盖异步 NRST、活动中同步 rst、正常状态转移、不同 A/B/CIN、输出屏蔽； M4 最终报告记录行为 51 个 checks、门级 43 个 checks（这是历史记录，不可套到新题）。



M4 报告依据课堂 FSM 重绘：状态、输出使能和异步 NRST

F03 [实遇] Lab 4 Slack -9.35 VIOLATED，但 setup 是 +99.94 MET

- `max_area = 0` 是教学脚本设的**零面积目标**，映射后的真实标准单元不可能同时面积为零，因此 `max_area` 可为负 slack。
- **这不是 setup timing 违例**。要看 violation 对应 `max_area`、`max_delay`、`setup`、`hold` 还是其它 constraint。
- **验收**：分别核对 `report_area`、`report_constraint -all_violators`（视版本参数）、`report_timing`；不要看到任意 `VIOLATED` 就乱加 pipeline、改时钟或者把老师设置删掉。

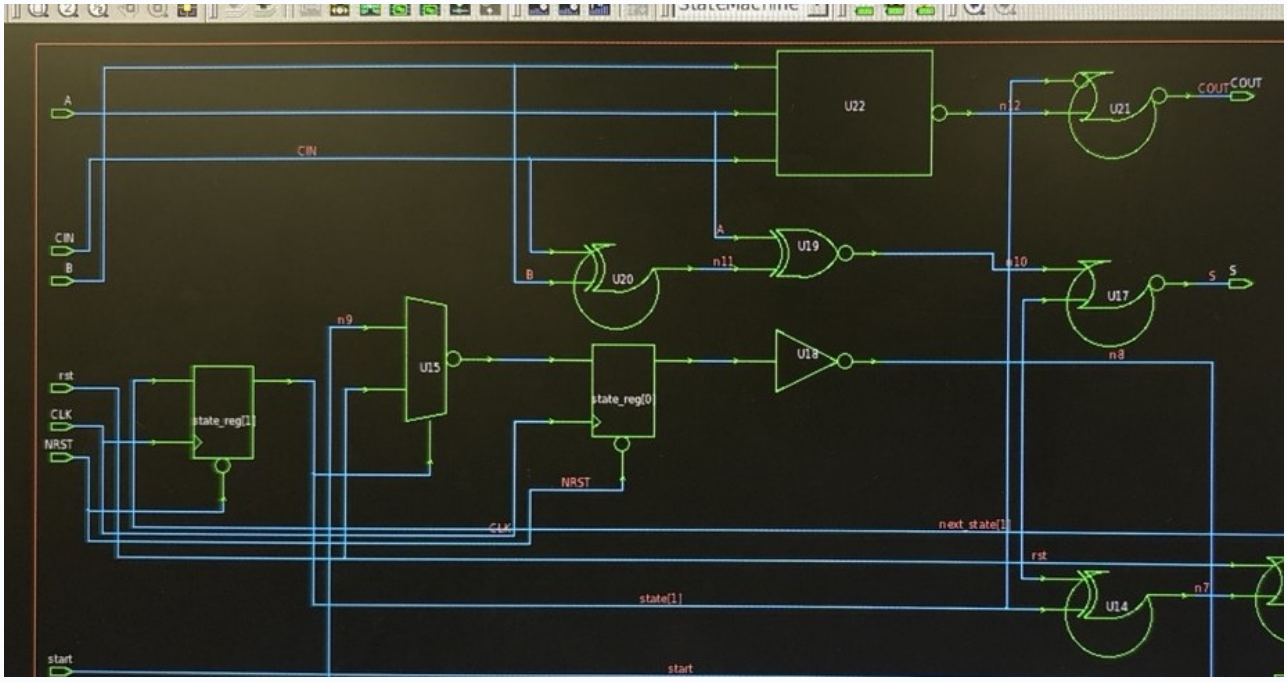


```
    input B,
    output S,
    output COUT
);

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  -bin/csh

[msc_tum_grp2@icd3pc19 Module4]$ mkdir -p Verification_Evidence
[msc_tum_grp2@icd3pc19 Module4]$ grep -ni -C 5 'TEST PASS' Behavioral/clean_r
un/sim.log
21-PASS @ 1352000 ns: start=0 rst=1 -> state=00
22-PASS @ 1452000 ns: start=1 rst=1 -> state=01
23-PASS @ 1552000 ns: start=0 rst=0 -> state=10
24-PASS @ 1652000 ns: start=0 rst=0 -> state=11
25-PASS: asynchronous NRST reset S3 -> S0
26:TEST PASS: 51 automatic checks
27-$finish called from file "../../tb/tb_StateMachine.sv", line 122.
28-$finish at simulation time              1664000
29-           V C S   S i m u l a t i o n   R e p o r t
30-Time: 1664000 ps
31-CPU Time:      0.420 seconds;       Data structure size:   0.0Mb
[msc_tum_grp2@icd3pc19 Module4]$ grep -niE -C 4 \
? 'POST-SYNTHESIS TEST PASS|Total checks|SDF annotation completed' \
? RTL/sim_gate.log
3-Contains Synopsys proprietary information.
4-Compiler version M-2017.03-SP2_Full64; Runtime version M-2017.03-SP2_Full64
;  Oct 25 13:34 2026
5-Doing SDF annotation ...... Done
6-VCD+ Writer M-2017.03-SP2_Full64 Copyright (c) 1991-2017 by Synopsys Inc.
7:POST-SYNTHESIS TEST PASS
8:Total checks = 43
9-$finish called from file "./tb_StateMachine_post.sv", line 182.
10-$finish at simulation time              1060000
11-           V C S   S i m u l a t i o n   R e p o r t
12-Time: 1060000 ps
[msc_tum_grp2@icd3pc19 Module4]$ grep -niE \
? 'Startpoint:|Endpoint:|data arrival time|data required time|slack' \
? Synthesis/logs/report.rpt
141:    Slack                   -6.51  (VIOLATED)
158:Endpoint                          Path Delay     Path Required     Slack
```

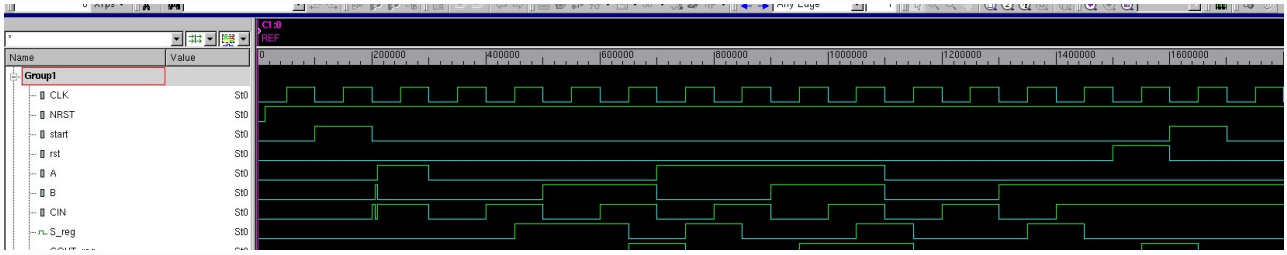
用户 M4 原始终端截图：报告中的约束 VIOLATED 与 Critical Path Slack +99.94 属于不同判断维度



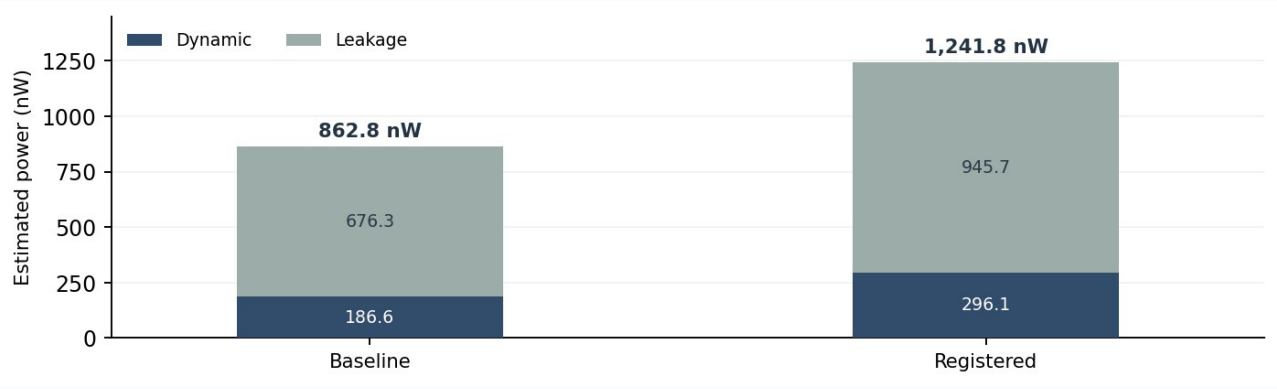
用户 M4 证据包：综合后的标准单元 Schematic（注意不是晶体管层级）

F04 [实遇] Lab 4 Optional：给输出加 FF 后测量的定义混乱

- 输出注册版 `StateMachineRegistered` 比原始版多两个 FF：12→14 leaf cells；FF 2→4；面积 6.514560→9.354240（+43.59%）；历史条件下总功耗 862.8274→1241.8026 nW（+43.92%）。
- **原因**：组合输出在时钟之间可能响应输入变化，而注册输出只在采样沿更新，但增加寄存器、延迟、时钟负载。
- **验收**：用相同激励/corner/时钟窗口和**两份有效 SAIF**进行对照；能证明输出注册后的波形稳定，**不等于**已经拍到了具体毛刺并证明完全消除每一种 glitch。



用户 M4 证据包：Registered 版本 DVE 时序；对比输出保持和周期延迟



用户 M4 报告：Original vs Registered 功耗对照重绘图（数值来自历史报告）

F05 [实遇] Lab 5 MatrixMultiplier 的 signed/宽度/valid 错误风险

- 目标为有符号 `1×2 · 2×1`，每次两个 8-bit signed 乘法，组合结果 `A1*B1 + A0*B0`，输出 **17-bit signed**。16-bit 产品进入 17-bit 累加前需符号扩展。
- 示例表达式（模块具体时序按实际 FSM）：

```
wire signed [15:0] product = $signed(A) * $signed(B);
wire signed [16:0] product_ext = {{1{product[15]}}, product};
```

- `START` 后持续运算；`OUT_STROBE` 表示**新 OUT 有效**，不能仅比较 OUT 而忽略 strobe、hold、reset、流式输入的先后拍数。
- 历史有效证据：行为与门级各 18/18 arithmetic checks；包含正负数、边界、异步复位。

F06 [实遇] Lab 5 的 7,287 cells / 7,035 buffers 不能不加分析地解释

- 历史综合报告：7,287 单元、约 7,035 buffer/inverter，cell area 1965.81（GF22 库面积单位）。
- **可能原因**：脚本中的 `set_fix_hold [all_clocks]`、hold uncertainty 与时序建模使 hold 修复引入大量 buffer；报告中对此仅作“合理推测”，**没有完成关闭 hold-fix 的对照实验**。
- **验收**：回看时序约束、单元统计、实际 hold fix；不要写“算法本身需要 7,000 个 buffer”或“已经证明根因”。

F07 [实遇] Lab 5 功耗时钟和综合时钟不同

- 综合 `create_clock -period 20ns` = 50 MHz；实际用于 SAIF 的 TB clock period = **10ns** ≈ 100 MHz。
- 历史 Power Compiler 报告总功耗 623.4345 µW（TT, 0.90 V, 25 °C，ZeroWLM）。
- **结论**：该数不能宣称“在 50 MHz 下测得”。要比较频率必须保持刺激/窗口可比且重跑 activity；动态功耗随活动和频率变，而 leakage 不简单按频率比例缩放。

---

4. T2 · Design Vision / 综合 / Tcl 最容易卡住的错误

E09 [实遇: Mock1] `script_path: Undefined variable` / `no such variable`

- **根因**：Tcl 变量没有初始化，或者 Design Vision 与系统终端不在同一工作目录。
- **优先修复（在 `design_vision>`，绝不是 csh）**：

```
cd /home/msc_tum_grp2/NM6008/Mock1/Synthesis
pwd
set script_path  [file normalize ./scripts]
set netlist_path [file normalize ./netlist]
set log_path     [file normalize ./logs]
set ddc_path     [file normalize ./ddc]
puts $script_path
puts [file exists "${script_path}/run_topdown.tcl"]
```

- **验收**：`pwd` 结尾为 `/Mock1/Synthesis`，`script_path` 结尾 `/Synthesis/scripts`，`file exists` 返回 `1`。如果返回 0，不准继续 `source`。

E10 `[实遇: Mock1]` `file exists=0` 指到了 `Behavioral/scripts` / `could not open ./scripts/...`

- **根因**：Linux 终端执行 `cd` 并不会让**已经运行中的 Design Vision** 自动切换 cwd。
- **修复**：在 `design_vision>` 执行 `pwd`，必要时 `cd <真实 Synthesis 绝对路径>`；修正 `script_path` 后再 `source "${script_path}/run_topdown.tcl"`。
- **验收**：`file exists` 为 1，后续脚本打开且没有因工作目录造成的 Missing file。

E11 `[实遇: Mock1]` `Current design is not defined (UID-4)`

- **根因**：读入 RTL/设置 Top 之前，配置中的 `set_fix_multiple_port_nets` 或其他命令操作当前设计。
- **修复顺序**：`read_verilog` → `current_design` → `link` → `check_design`，然后才执行需要设计对象的设置。不要直接整份重写 `.synopsys_dc.setup`，也不要无条件反复 source 它。
- **验收**：`current_design` 有效，`check_design` 无致命问题。

E12 `[实遇: Mock1]` Top 名称不匹配：`secretcode` VS `secret_code_four`

- **根因**：实际 RTL 声明 `module secret_code_four`，旧 Tcl 却引用 `secretcode`。
- **修复**：核对三处：`scripts/ReadVerilogFiles.tcl`（源文件列表）、`scripts/run_topdown.tcl`（指向设计脚本）、`scripts/secretcode.tcl`（`current_design`、设计名）。已知本例：

```
# ReadVerilogFiles.tcl
read_verilog ./sources/digit_counter.v
read_verilog ./sources/secret_code_four.v

# run_topdown.tcl
source "${script_path}/ReadVerilogFiles.tcl"
source "${script_path}/secretcode.tcl"

# secretcode.tcl (片段)
set design "secret_code_four"
current_design secret_code_four
```

- **验收**：`link; check_design; report_reference` 均针对正确 Top；Single/Four 两种设计分别生成命名不同的网表、报告，防止覆盖。

E13 `[实遇/手册]` 综合结束却找不到 `.sdc` / `.v` / `.sdf`

- **定位**：脚本是否有 `write_sdc`、`write_sdf`、`write -format verilog`；设计是否 compile；输出的**相对路径基于 Design Vision 当前 cwd**；文件可能在另一目录或没执行到导出。
- **检查**（Linux）：

```
pwd
ls -lah netlist logs ddc
```

```
grep -RniE 'write_sdc|write_sdf|write.*verilog|write.*ddc' scripts
find . -type f \( -name '*.sdf' -o -name '*.sdc' -o -name 'Syn_*.v' \) -print
```

- **如需要且脚本没有导出**：在 Design Vision 确认 Top 和 compile 后：

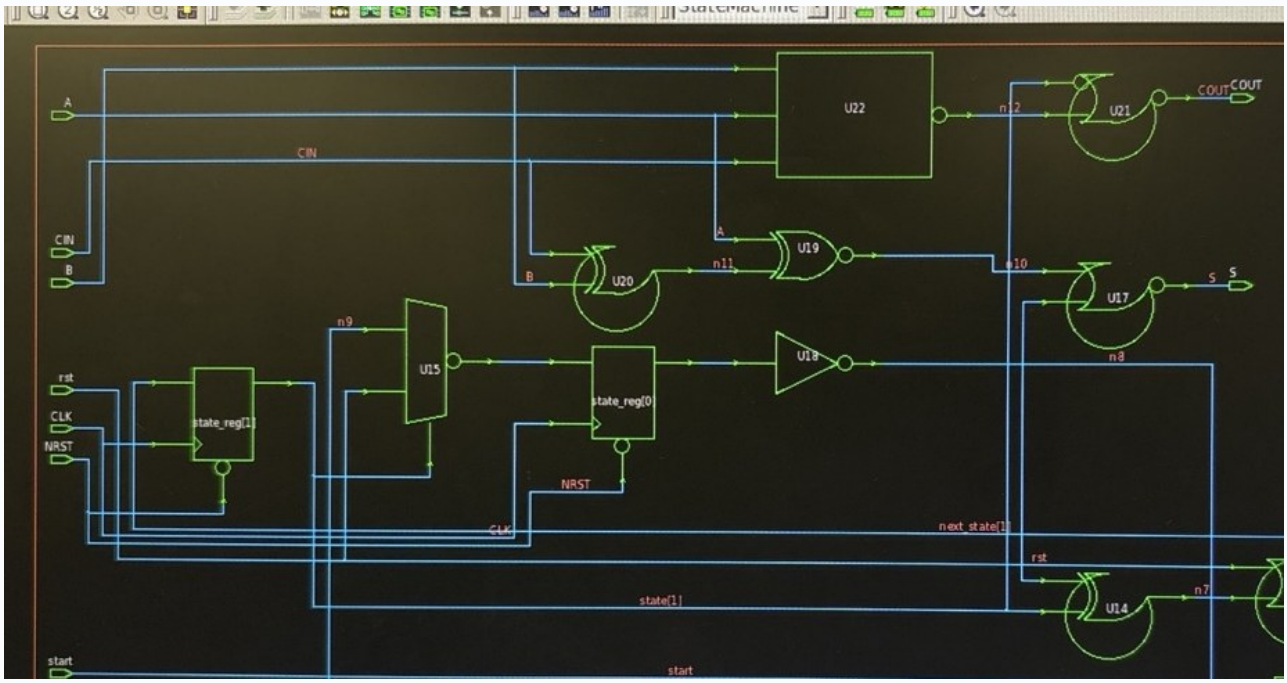
```
write -format verilog -hierarchy -output ./netlist/Syn_TOP.v
write_sdf ./netlist/Syn_TOP.sdf
write_sdc ./netlist/Syn_TOP.sdc
write -format ddc -hierarchy -output ./ddc/TOP.ddc
```

将 TOP 替换为真正模块名；`write_sdc` 是否需要取决于课程要求及约束。**M3/课程脚本的 SDC 输出以本轮实际脚本为准；Mock1 增补 SOP 含 `write_sdc`，故应检查是否执行。 `.sdc` 不参与 VCS 门级功能编译的 `-f` 文件列表。**

- **验收**：文件非空、更新时间在本轮之后、Top/内容不是 FullAdder 旧产物；报告不是旧 `report.rpt`。

E14 `[实遇]` Schematic 只看到大方框、子模块，看不到 MOS 管

- **解释**：`Design Vision Schematic` 可能先显示保留层次；进入 `M1/digit_counter_1` 才可看到内部逻辑。已完成 `compile` 后仍可能显示 hierarchy。**单元级电路≠晶体管级电路。**
- **执行**：`current_design <TOP>; link; compile; report_reference; report_cell; report_area` → GUI 双击层次模块向下展开。
- **验收**：看到了 GF22 standard cells / FF / MUX 等 mapped elements；若目标是 PMOS/NMOS，需另用 Virtuoso/schematic 或找到相应 cell 的 SPICE/CDL，而不是在 DV 里继续缩放。



用户 M4 证据包：映射后的层级/单元连接图；Design Vision 不展示晶体管细节

E15 `[实遇/预防]` `No clocks in this design` / `Clock not found`

- **Lab 3**：FullAdder 是无时钟组合逻辑，不能为了消警告凭空加 clk。
- **Lab 4/5/Mock1**：应当有实际时钟端口，检查 `get_ports CLK` vs `get_ports clk`、`create_clock` 是否真正执行。
- **验收**：`report_clock` 有预期时钟（时序题）；检查 `report_timing -delay_type max -max_paths 5`。**`-max_paths 5` 与 `10` 都是允许的报告显示条数，不是时钟参数或综合正确性的条件；5 是之前 Mock1 SOP 示例值。**

E16 [实遇/预防] 综合 report 出现 VIOLATED，但不知道坏在哪里

- **方法**：先找到约束名称，区分 `max_area`、setup、hold、max_transition、max_capacitance、max_fanout。
- **重要公式**：setup slack = required time − arrival time；**正值说明该路径在当前约束下满足，负值说明该路径违例**。不同 report 的 slack 不能横向当成同一种指标。
- **命令**：`report_timing -delay_type max -max_paths 5`、`report_timing -delay_type min`（按版本支持）、`report_area`、`report_constraint -all_violators`（先 `help report_constraint` 核对可用选项）。
- **验收**：能指出具体 startpoint、endpoint、路径类型、arrival/required/slack 和对应 constraint，而非只说“delay 是 10ns”。

E17 [预防] 改了 Behavioral RTL，综合结果却不变

- **根因**：Synthesis 的 `sources/` 还是旧文件；`ReadVerilogFiles.tcl` 指错；已有 `.ddc` 或上一轮 netlist 被当作新结果。
- **修复**：`cmp Behavioral/Top.v Synthesis/sources/Top.v` 或 `sha256sum` 对比；确认 `read_verilog` 列表、Top、时间戳；拷最新 RTL 后重跑 T2。
- **验收**：`netlist/` 新时间戳、关键逻辑与 RTL 一致、`report_reference` 是 GF22 而非仅 RTL hierarchy。

E18 [预防] 工艺库找不到 / link unresolved / GF22 cell 缺失

- **根因**：`.synopsys_dc.setup`、`common_setup_gf22.tcl` 的 search_path、target_library/link_library 配置错误；相对路径随 cwd 变化。
- **修复**：检查脚本和真实 `.db` 位置：`ls -lah Library/; grep -nE 'search_path|target_library|link_library' .synopsys_dc.setup common_setup_gf22.tcl`；修正**本机**相对/绝对路径后 `link; check_design`。
- **验收**：GF22 cell references 实际解析，输出网表具标准单元实例。不得把 `.db` 当作 VCS 可编译 Verilog 模型。

---

5. T2 正确操作顺序（避免反复踩 E09–E18）

1. 在**真实** `Synthesis` 目录启动 Design Vision（必要时在 GUI `pwd`、`cd`）。
2. 检查 `.synopsys_dc.setup`、`common_setup_gf22.tcl`、`scripts/MyDefaults.con`，只调整必须改变的路径/约束。
3. 确认 `sources/` 是最新版 RTL；`scripts/ReadVerilogFiles.tcl` 列齐所有子模块；`run_topdown.tcl` 调到当前 Top 对应脚本。
4. `read_verilog` → `current_design` → `link` → `check_design` → 正确 clock/I/O constraints → `compile`。
5. `report_reference` / `report_cell` / `report_area` / `report_clock` / `report_timing`，**分析违例类型**。
6. 导出 `Syn_TOP.v`、`Syn_TOP.sdf`、可选 `Syn_TOP.sdc`、DDC 和 reports；核实内容、时间戳与目录。
7. 才进入 T3。GUI 能看到方框不等于已完成映射。

---

6. T3 · GF22 门级仿真 + SDF 常见问题

E19 [实遇] library.f: No such file / 找不到 std-cell module / UDP

- **根因**：`rtl`/`RTL` 大小写或 `-f` 相对目录错；`library.f` 内引用的 GF22 `.v` 模型不存在或不适合当前运行目录。
- **修复**：

```
pwd
find .. -name library.f -print
cat ./library.f
```

检查文件里的每条 `-v <library_cell_model>.v` 路径。用 `vcs -debug +v2k +neg_tchk ./TB.v ./Syn_TOP.v -f ./library.f -l compile_gate.log` 编译（替换文件名）。

- **验收**：所有 cell 可 elaborate，VCS 无致命 missing module；不要把设计 `.db` 输入给 VCS。

E20 [实遇] SDF annotation warning / `SDF instance not found` / delay 似乎没生效

- **根因**：`$sdf_annotate` 的实例不是 TB 真实 DUT 名；SDF/netlist 与本轮 Top、层次不同；未使用对应版本的 SDF。
- **检查/修复**：

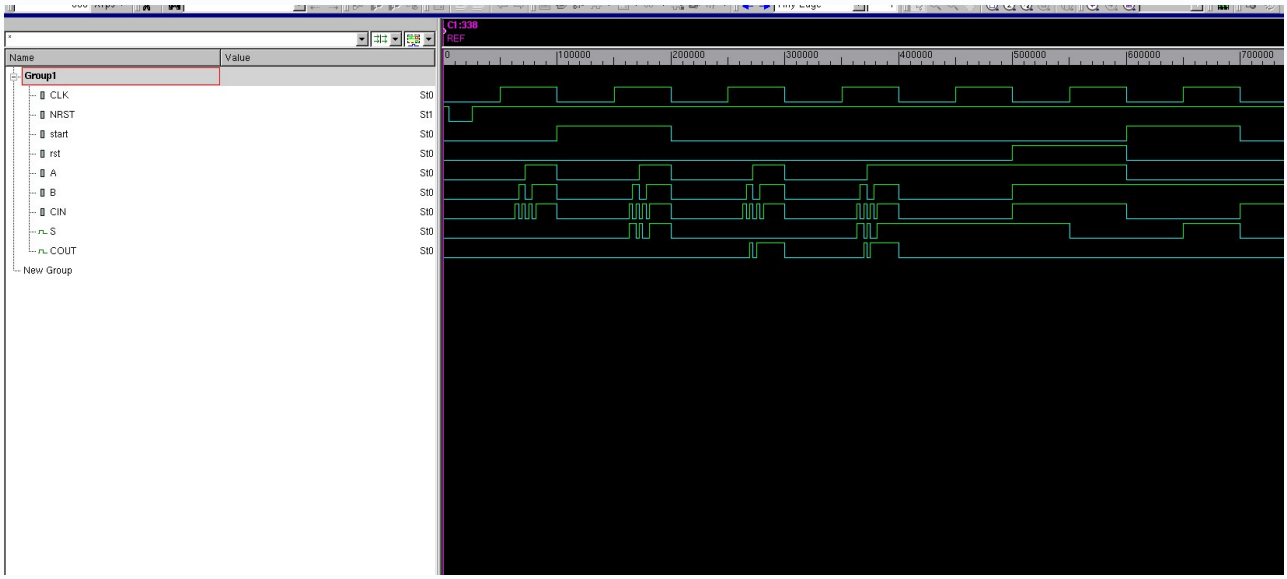
```
`timescale 1ns/1ps
// 在 TB 中:
// actual DUT instance name is dut
initial $sdf_annotate("Syn_TOP.sdf", dut);
```

查 `grep -niE 'SDF|annotat|SDFCOM|error|warning' sim_gate.log`，对照 SDF 的 cell instance path 和 netlist 实例。

- **验收**：日志显示 SDF 读入、注释目标和警告数；不能仅因有波形就声称 SDF 完全成功。

E21 [实遇] `SDFCOM_CFTC` / 负 timing-check limit (`WSUM`) 警告

- **实际记录**：M4 有 conditional `$hold` 不匹配及负 limit 修正；M5 有 **54 条**警告（含 `SDFCOM_CFTC` 和 `WSUM`），但门级 directed arithmetic tests 为 18/18 PASS。
- **处理**：区分功能 simulation pass、SDF delay 注释成功、**timing check 覆盖完整**三件事。搜 SDF warning 的发生对象和数量；`+neg_tchk` 是手册给的兼容选项，不是 STA signoff 证明。
- **验收**：报告忠实说明警告与功能验证边界，不写“timing clean/无警告”。



用户 M4 证据包：门级 DVE 波形。波形存在不等于所有 SDF timing check 通过

E22 [实遇] 门级输出 X / 输出过早比较 / 仿真波形出现窄脉冲

- **定位**：先确认 async reset、初始 state、DUT 时钟、TB 采样时刻、是否刚好踩到 SDF propagation/hold time。
- **修复**：对序列输入在安全边缘驱动；在 spec 允许时的有效窗口采样；用 DVE 放大 ns 甚至 ps 局部波形。
- **验收**：能够说清是 bug、采样竞争还是合法 gate hazard；不要靠删 timing check、无条件屏蔽 X 来造 PASS。

E23 [实遇] 找不到 inter.vpd，也就做不了功耗

- **检查**：TB 有 `$vcdplusfile("inter.vpd")` / `$vcdpluson(...)` / `$vcdplusoff(...)`；是否**运行** simv；文件是否生成在当前 cwd。
- **修复**：`find . -name '*.vpd' -print`；`./simv -l sim_gate.log`；`ls -lh *.vpd`。
- **验收**：新生成、非空的 VPD 且激励/仿真范围正确；再进入 T4。

E24 [实遇/手册] RTL VS rtl 以及 $sdf_annotate 三个 TB 修改点

M3 文档 P16 有大写 `RTL` 表述，后续 P18 使用小写 `rtl`。**Linux 大小写敏感，一律以真实 `ls` 为准**。P17 要求 T3 TB 至少注意：`timescale 1ns/1ps`、`$sdf_annotate`、`$vcdpluson/off`；T3 用 `library.f` 和 GF22 模型，不是把 T1 的 RTL 当 Gate netlist。

---

7. T4 • VPD → VCD → SAIF → Power Compiler

7.1 最小正确命令链（以 M3 P19–20 为基准）

Linux/csh（进入实际 `rtl/PowerAnalysis`；先检查 GF22 `.db` 搜索路径）：

```
csh
source ../../Synthesis/synopsys_linux_vR_2020_cshrc
ls -lh ../inter.vpd
vpd2vcd ../inter.vpd inter.vcd
vcd2saif -i ./inter.vcd -o inter.saif
ls -lh inter.vcd inter.saif
dc_shell-xg-t
```

进入 `dc_shell>` 后（以下 FullAdder 是 M3 示例，不可以直接套在 M4/M5/Mock1）：

```
read_verilog ../Syn_FullAdder.v
current_design FullAdder
link
read_saif -input ./inter.saif -instance FullAdder_tb/U1
report_saif -hierarchy
report_saif -missing
report_power > rpt_power.txt
report_power -hier > rpt_power_hier.txt
report_power -hier -nosplit > rpt_power_hier_nosplit.txt
report_power -cell > rpt_power_cell.txt
```

注：`report_saif` 可用参数依版本而异，若提示 unknown command / option，先 `help report_saif`。老师 M3 手册 P20 用 `>>` 追加：反复运行可能把两轮结果混在一起；用 `>` 会覆盖，先备份旧证据。

E25 [实遇: Lab4 Optional] registered.vpd doesn't exist → registered.vcd / .saif 连环缺失

- **根因**：Registered TB 没生成 VPD、dump 名字错、simulation 没真正 run，或错误 cwd。
- **修复**：先跑本轮 registered gate sim，检查 `registered.vpd`；**必须 VPD 非空**才能 `vpd2vcd` → `vcd2saif`。
- **验收**：`ls -lh registered.vpd registered.vcd registered.saif` 均存在且新鲜，再读 SAIF。**若 PWR-201 仍报错，不允许用工具的默认活动功耗充作 Registered 的实测结果**。

E26 [实遇] PWR-201 Cannot find SAIF file

- **根因**：前面转换失败/写在别的 cwd/文件名不一致。
- **修复**：`pwd; ls -lh inter.saif; find .. -name '*.saif' -print`；从 VPD 一步一步检查，每步失败即停止。

- **验收**：`read_saif` 成功，`report_saif` 显示预期对象得到 activity annotation。

E27 `[实遇]` `report_saif` 0% / 功耗报告竟仍有数字

- **根因**：`read_saif -instance` 的 `TB/U1` 层次错；Power Compiler 可能用默认 switching activity 给出**估计值**。
- **修复**：检查 TB 的真实模块名和 DUT 实例名；在 VPD/VCD/SAIF 中确认 scope；设 `read_saif -input ... -instance tb_StateMachine_post/dut` 这类**与本设计相符的实际路径**。
- **验收**：目标 nets/ports/pins 覆盖率有效；对比实验最好达到目标 mapped objects 完全覆盖且检查 missing。无有效注释时不得报告为“基于门级活动的功耗”。

```
Operating Conditions: TT_0P90V_0P00V_0P00V_0P00V_25C   Library:
gf22nsdvlogl20hdl116a_TT_0P90V_0P00V_0P00V_0P00V_25C
Wire Load Model Mode: enclosed

Design        Wire Load Model            Library
------------------------------------------------
StateMachine           ZeroWLM           gf22nsdvlogl20hdl116a_TT_0P90V_0P00V_0P00V_0P00V_25C


Global Operating Voltage = 0.9
Power-specific unit information :
    Voltage Units = 1V
    Capacitance Units = 1.000000pf
    Time Units = 1ns
    Dynamic Power Units = 1mW    (derived from V,C,T units)
    Leakage Power Units = 1nW


  Cell Internal Power  = 150.8783 nW   (81%)
  Net Switching Power  =  35.5967 nW   (19%)
                         ---------
Total Dynamic Power    = 186.4750 nW  (100%)

Cell Leakage Power     = 648.1411 nW


                 Internal         Switching           Leakage            Total
Power Group      Power            Power               Power              Power   (   %    )  Attrs
--------------------------------------------------------------------------------------------------
io_pad             0.0000           0.0000              0.0000             0.0000  (   0.00%)
memory             0.0000           0.0000              0.0000             0.0000  (   0.00%)
black_box          0.0000           0.0000              0.0000             0.0000  (   0.00%)
clock_network      0.0000           0.0000              0.0000             0.0000  (   0.00%)
register       9.0165e-05       1.0109e-05          273.7317           3.7401e-04  (  44.81%)
sequential         0.0000           0.0000              0.0000             0.0000  (   0.00%)
combinational  6.0713e-05       2.5488e-05          374.4094           4.6061e-04  (  55.19%)
--------------------------------------------------------------------------------------------------
```

用户 M4 证据包：实际 Power Compiler 输出截图。要结合 read_saif 的 annotation 状态理解

E28 `[实遇：M5]` 100% activity 但仍有 `PWR-452 1935 unmatched`

- **实际数据**：M5 target-design 活动覆盖 nets **7,347**、ports **37**、pins **15,128** 均标为 100%；同时 SAIF 源文件有 **1,935 个对象无法匹配**。
- **判断**：这不必然矛盾：一个是映射后目标对象的覆盖率，另一个是源 SAIF 中多余/优化前对象。应分析 unmatched 来源，不能称完全无注释问题或完全功耗 signoff。
- **验收**：同时保存 `read_saif` warning 和 `report_saif` 报告，忠实说明边界。

E29 `[实遇/预防]` `report_switching_activity: unknown command`

- **处理**：当前 DC 版本未提供该命令，先 `help`；使用版本支持的 `report_saif -hierarchy`、`report_saif -missing` 等实际存在的命令，不要为消错去修改 PDK。

E30 `[实遇/预防]` 功耗报告文件在哪里？`report_power >> file` 混了很多轮

- **修复**：在 DC Tcl `pwd` 查输出 cwd；Linux `find . -name 'rpt_power*'` 查文件，报告头部核对时间、Top、corner、frequency、activity annotation。
- **验收**：报告是本轮输入和本轮 SAIF 的结果；不同 clock/corner/vector 的数值不能直接比。

专题 B｜后端：Lab 6 Innovus 物理实现与 RC 提取

8. T5 · Lab 6 Innovus 版图流程与实际故障

> **注意**：Lab 6 使用 Cadence Innovus **20.12（历史操作记录）** 与 GF22 FDSOI。它是物理设计；不要把 Design Vision 的 schematic、理想 WLM 时序、Innovus 物理 layout、晶体管级 schematic 混为一谈。2026 手册与老师现场修订存在区别，优先以老师更新要求为准。

8.1 正确主流程（M6 §2–3，对应 p3–21）

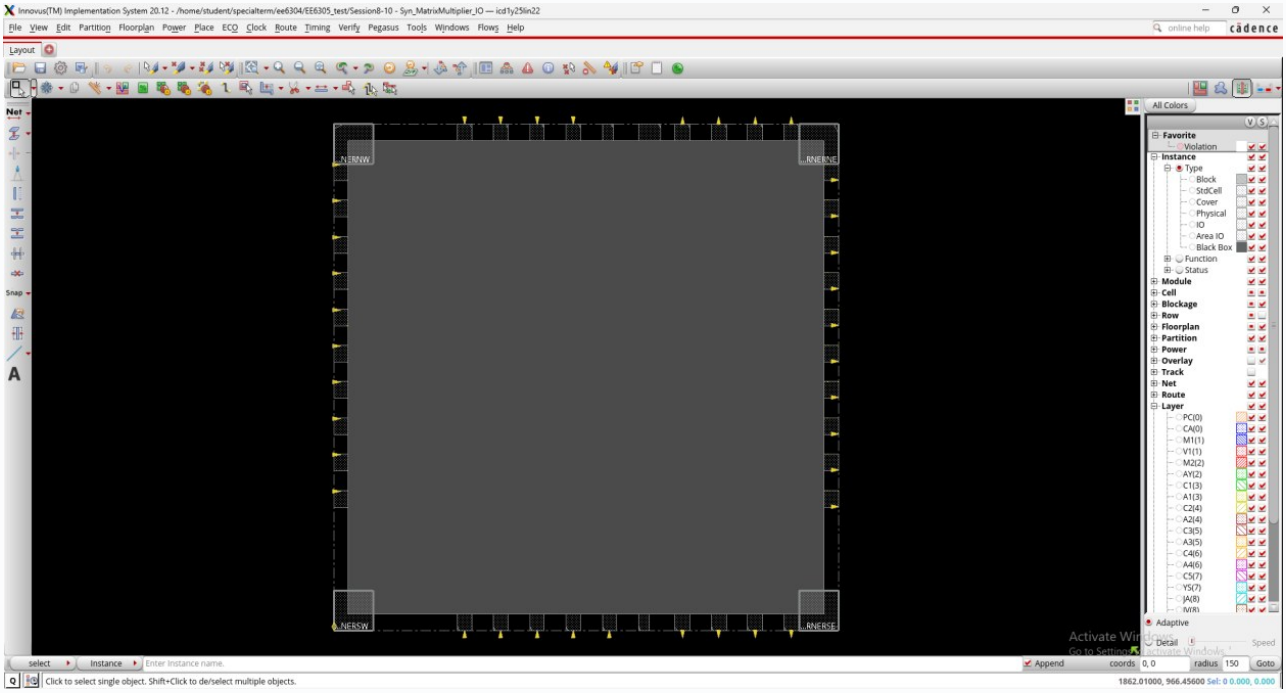
1. **启动**：在 Module6 目录 `csh; source cshrc_inno; which innovus; innovus`（`source` 在 csh 环境）。
2. **设计输入**（Innovus Console）：`source Syn_MatrixMultiplier_IO.globals` → `init_design`。
3. **PG 连接**：

```
clearGlobalNets
globalNetConnect vdd -type pgpin -pin VDD -inst *
globalNetConnect gnd -type pgpin -pin VSS -inst *
```

4. **Floorplan**：手册演示 `floorPlan -site GF22_SST -d 1400 1400 150 150 150 150 -noSnapToGrid`；这些是课程示例，不是其它设计的通用规则。
5. **Power Ring**：用老师脚本 `setAddRingMode` / `addRing`，core ring 采用 C1/C2 等指定工艺层。
6. **Special Route**：`setSrouteMode` → `sroute`，**层名见 E32**。
7. **IO Pad Filler**：由大到小填充；End Cap；然后 **Standard-cell Placement**：`placeDesign`。
8. **Pre-CTS**：`timeDesign -preCTS ...`，必要时 `optDesign -preCTS`。
9. **CTS**：`create_ccopt_clock_tree_spec` → `ccopt_design -prefix cts`；检查 post-CTS setup 和 hold。
10. **Detailed Route**：按手册 `source ./scripts/Route.tcl`，检查 post-route STA（onChipVariation / RC setup）和 DRC。
11. **Core filler + DRC/ECO**：依手册 `addFiller ... -doDRC`，修复后再次 `verify_drc`（见 E35）。
12. **导出**：`streamOut` 产生 GDSII；`write_sdf` 产生 SDF；`saveNetlist` 产生 post-layout netlist。SPEF 可尝试由 `rcOut` 输出，但**本轮老师豁免了未成功的高精度寄生提取**；preRoute 估算与 postRoute signoff 提取不能混淆（见 E36/E47–E51）。

E31 `[实遇]` `init_design` 后很多 Warning（约 2890 条）

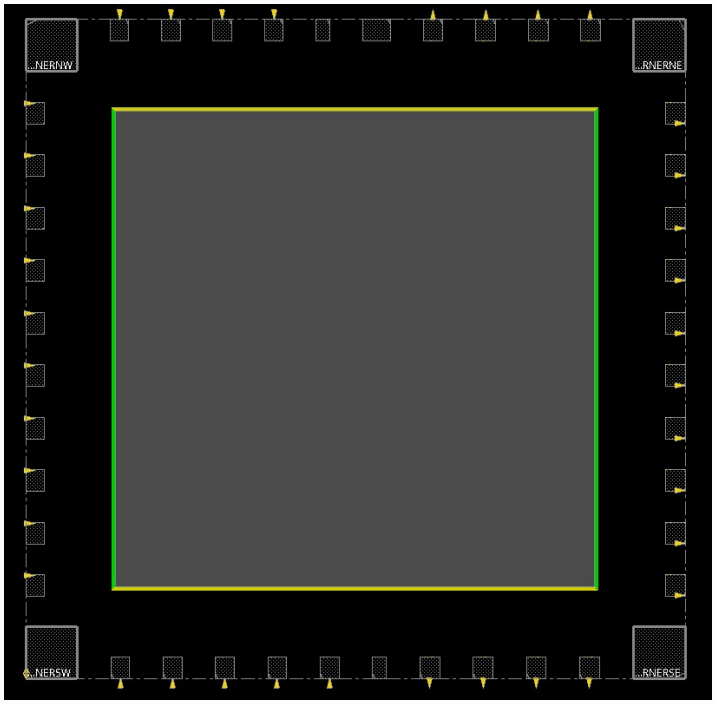
- **历史现象**：初始 design 可见、0 Errors，约 2890 Warnings（包括 Time Units Inconsistency、Skipped Unsupported Command 等）。
- **处理**：首先找 fatal Errors、未加载库/ports/unresolved 和具体后果；warning 数字本身不能证明流程失败或成功，也不能一键全部忽略。
- **验收**：设计的 `Syn_MatrixMultiplier_IO`、core/pads、PG nets/约束正确，后续工具能进行 place/route；重要 warning 有解释。



M6 官方手册：Innovus 设计导入界面示例，便于核对 IO pad/core 是否出现

E32 [实遇/教师更新] Special Route 找不到 `MET1(1)` / `MET4(4)`

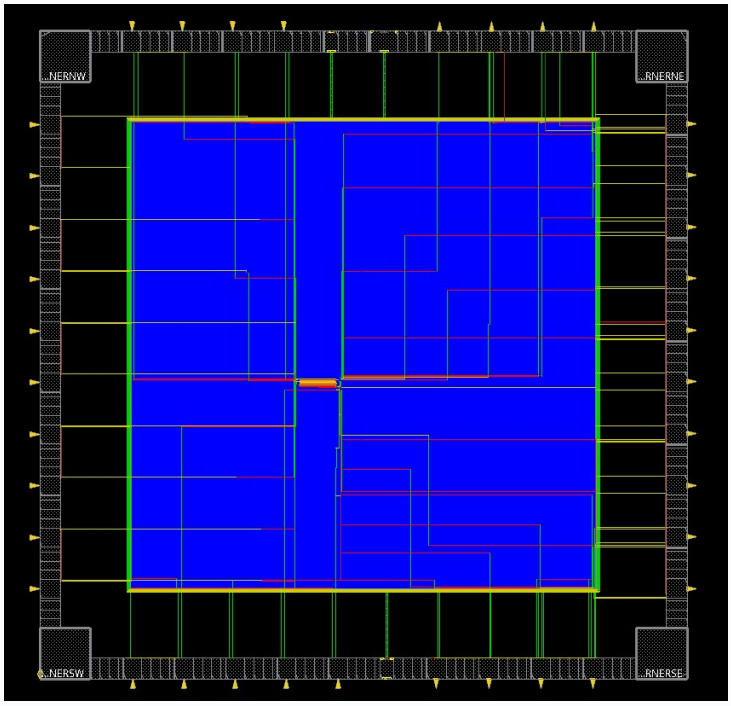
- **根因**：原版 M6 P10 采用的层标识与当前课堂工艺配置不一致。
- **修复**：**老师现场更新**：Special Route layer-change range 改用 `M1(1)` 到 `C2(4)`，而**不是**原版手册 `MET1(1)` / `MET4(4)`。检查并修改 `sroute` 对应 layer range 和 via range，保留老师配置的其他工艺参数。
- **验收**：Special Route 完成，PG 网络连续，没有意外 short/open；不能因 GUI 出现金属线就忽略后续 DRC。



M6 官方手册：Special Route 操作参考；**图中的旧 layer 名必须按 2026 教师修订替换**

E33 [实遇/手册] Power Ring/SRoute 创建了，但界面太乱、看不到 Cell Placement

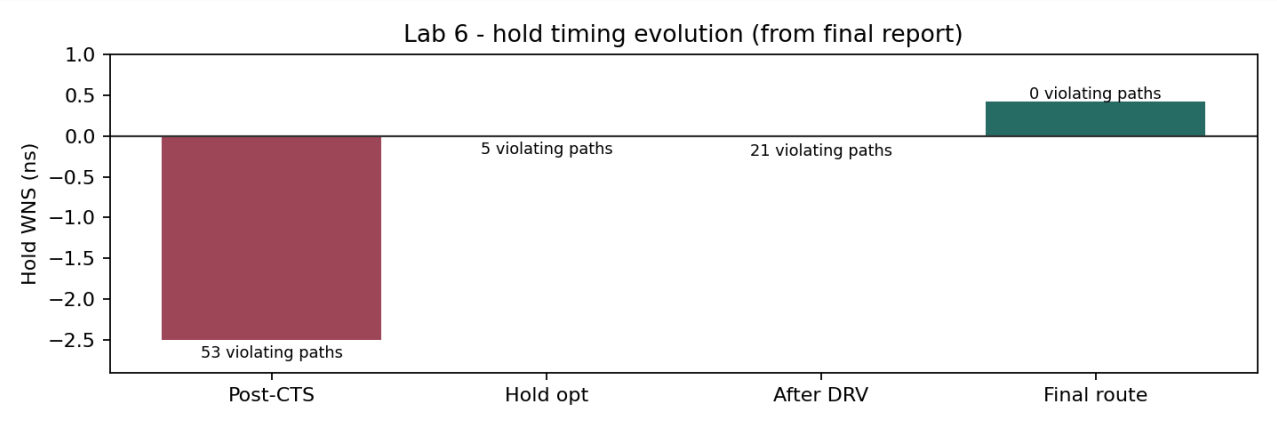
- **根因**：几十/几百层可视对象叠加、Routing 遮住标准单元；有时实际是**显示过滤**而非布局不存在。
- **修复**：在 Innovus Display/Layer 中有选择地关掉部分 Routing/Metal，保留 `StdCell/Instance`、Rows、Core/Die 及必要 PG 结构；用 Zoom to Fit 或框选 Core。保留一个**全部 routing 可见**的全局布局证据和一个**隐藏 metal** 的放置细节证据。
- **验收**：截图可明确识别对象种类，**隐藏导线不会删除数据库中的 routing**；截图题注必须写清 display-filtered。



M6 官方手册：Standard Cell placement / GUI 显示参考；关闭层显示不等于改变数据库

E34 [实遇] post-CTS Hold WNS 负值，修过后又变差

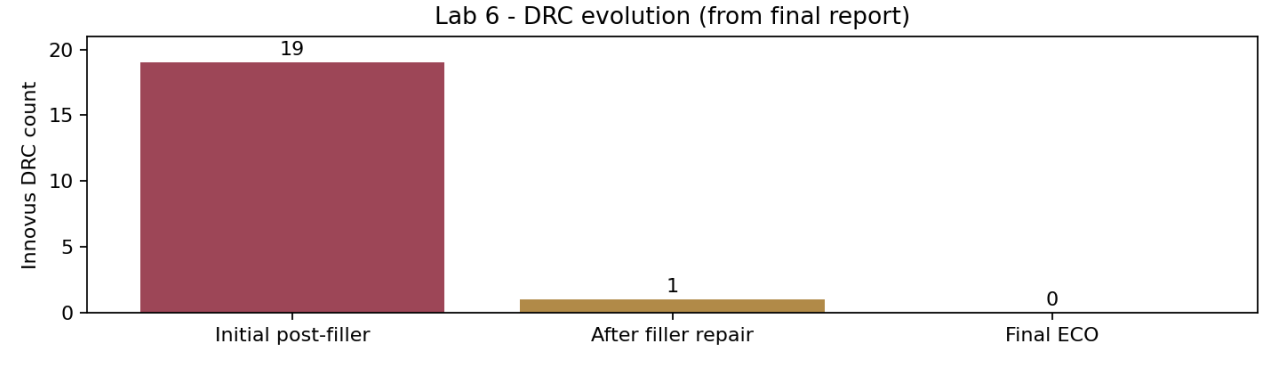
- **实际历史**：M6 post-CTS hold 初始 **−2.492 ns / 53** paths；hold optimization 后 **−0.001 ns / 5**；额外 DRV optimization 后 **−0.014 ns / 21**；最终 routed **+0.419 ns / 0**。
- **根因/原理**：hold 需要最小数据路径满足约束；短路径、clock skew、2.5 ns clock uncertainty 等会加剧问题。hold fix 往往插 buffer/delay 和增加路由，可能损伤 setup/面积。
- **处理**：在 CTS 后对 **hold** 模式做优化，**每次优化后**重新 `timeDesign` 检查 WNS/TNS/violating paths；查看实际 SDC、是否将 delay cell 标为 `dont_use`；必要时由工具进行 incremental ECO。**不能简单把 uncertainty 改成 0 来伪造 timing clean**。
- **验收**：同一最终 routed database 下 setup/hold 均无 violating paths，而非拿上一步不同 database 的截图互相拼接。



根据最终 M6 实验报告重绘：Hold WNS 和违反路径数在多个阶段反复变化

E35 [实遇] 插 Core Filler 后 DRC 出现 19 个违例，修到 1 个 M1 short，最终 0

- **实际历史**：第一次 post-filler `verify_drc` **19** 个 violations，部分与 M1 short/spacing 有关；filler 修复后 **1** 个 M1 short（routed net 与标准单元 pin）；incremental ECO route 后最终 **0**。
- **查错顺序**：保存 DRC report/markers → 选中 marker 确认 layer/net/cell、short 还是 spacing → 确定是否 filler 尺寸、特定 routed segment 与 pin 相撞 → 在对应操作层进行最小 ECO/route 修复 → **重新运行** `verify_drc`。
- **验收**：最终报告明确 `No DRC violations were found`。**仅仅关闭 DRC 显示、隐藏 M1、重新截图，绝对不算修复**。
- **范围说明**：这是 Innovus 内建 DRC check 清零，不等同于 Foundry 签核 DRC/LVS 全部完成。



根据最终 M6 报告重绘：DRC 数量 19→1→0；最后必须再次 verify_drc

E36 [实遇/教师豁免] Quantus 报 `GLIBC_2.25/2.27/2.28 not found` / SPEF 提取失败

- **根因**：所调用 Quantus 24.10 可执行程序依赖的 GLIBC 高于实验室 CentOS 7 提供的版本，这是二进制/系统兼容性问题，不是设计 Verilog 错误。
- **处理**：保存终端错误证据，不要私改系统 GLIBC 或声称 `.spef` 生成成功；2026 本轮老师**已明确豁免寄生参数提取步骤**。
- **验收**：报告写“attempted but not completed / instructor waived”；SDF/GDS/netlist 仍需按要求核实。其它年份课程不得自动沿用豁免。

E37 [实遇] `100% with Fillers` 被误当成 100% logic utilization

- **解释**：历史报告 functional-cell density 约 **0.327%**（不含 filler），`100% with Fillers` 代表包含 physical filler 的填充，不是逻辑单元占据了 100% Core。
- **验收**：报告分别写 functional density 与 including fillers；不要基于填充率指责 placer 失效。

E38 [实遇/手册] 导出 GDS/SDF/Netlist 后路径、Top 或数据不一致

- **手册命令**（M6 P20–21，以下文件名是历史 Top 示例）：

```
streamOut Syn_MatrixMultiplier_IO.gds2 -mapFile GDS/
22FDSOI_10M_2Mx_5Cx_1Jx_2Qx_LB_edi2gds_colored.layermap -libName DesignLib -structureName
Syn_MatrixMultiplier_IO -units 1000 -mode ALL
write_sdf Syn_MatrixMultiplier_IO.sdf
saveNetlist output_netlist/Syn_MatrixMultiplier_IO.v
```

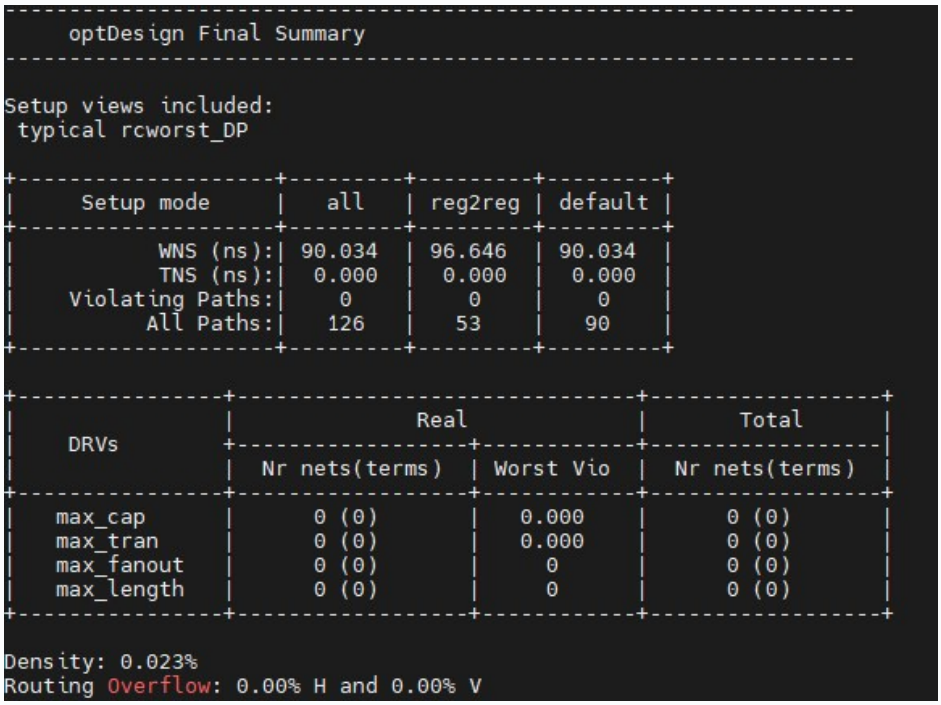
  注意：以上单行 `streamOut` 应在 Innovus Console 输入，mapFile 必须实际存在；GDS/SDF/netlist 的 Top 与布局 DB 匹配。
- **历史已核实**：GDSII 约 **22 MB**，SDF 约 **3.7 MB**，netlist 约 **1.3 MB**，顶层为 `Syn_MatrixMultiplier_IO`。
- **验收**：`ls -lh` 显示非空、新文件；netlist 检查顶层声明：`grep -n '^module Syn_MatrixMultiplier_IO' output_netlist/Syn_MatrixMultiplier_IO.v | head`。文件大小不是通用及格标准。



M6 官方手册：GDS/SDF/netlist 导出操作示例；文件名与 mapFile 需和实际 Top 匹配

E39 [实遇] post-route Setup/Hold 结果如何判断能否提交

- **历史最终 Innovus timing（所用 RC 模型有限制）**：Setup WNS **+87.329 ns**；Hold WNS **+0.419 ns**；双方 TNS **0**、violating paths **0**；real electrical DRV violations **0**；final `verify_drc` **0**。
- **要区分**：`timeDesign -signoff` 是**工具命令/分析模式**，不代表已经完成独立 signoff extraction、foundry DRC/LVS 或全 PVT corner verification。
- **验收**：保存 **同一最终 DB** 的 setup/hold/DRV/DRC 证据与导出文件；报告写“Lab-level Innovus clean”，不写“foundry fully signed off”。



```
------------------------------------------------------------------
     optDesign Final Summary
------------------------------------------------------------------

Setup views included:
 typical rcworst_DP

+--------------------+---------+---------+---------+
|     Setup mode     |   all   | reg2reg | default |
+--------------------+---------+---------+---------+
|           WNS (ns):| 90.034  | 96.646  | 90.034  |
|           TNS (ns):|  0.000  |  0.000  |  0.000  |
|    Violating Paths:|    0    |    0    |    0    |
|          All Paths:|   126   |   53    |   90    |
+--------------------+---------+---------+---------+

+----------------+-----------------------------------+------------------+
|                |              Real                 |       Total      |
|    DRVs        +------------------+----------------+------------------|
|                |  Nr nets(terms)  | Worst Vio      |  Nr nets(terms)  |
+----------------+------------------+----------------+------------------+
|   max_cap      |      0 (0)       |   0.000        |      0 (0)       |
|   max_tran     |      0 (0)       |   0.000        |      0 (0)       |
|   max_fanout   |      0 (0)       |     0          |      0 (0)       |
|   max_length   |      0 (0)       |     0          |      0 (0)       |
+----------------+------------------+----------------+------------------+

Density: 0.023%
Routing Overflow: 0.00% H and 0.00% V
```

M6 官方手册：时序报告 GUI 示例。报告截图不是我们最终数值的原图

E40 [预防/手册] Lab 6 的 screenshot 该怎么拍

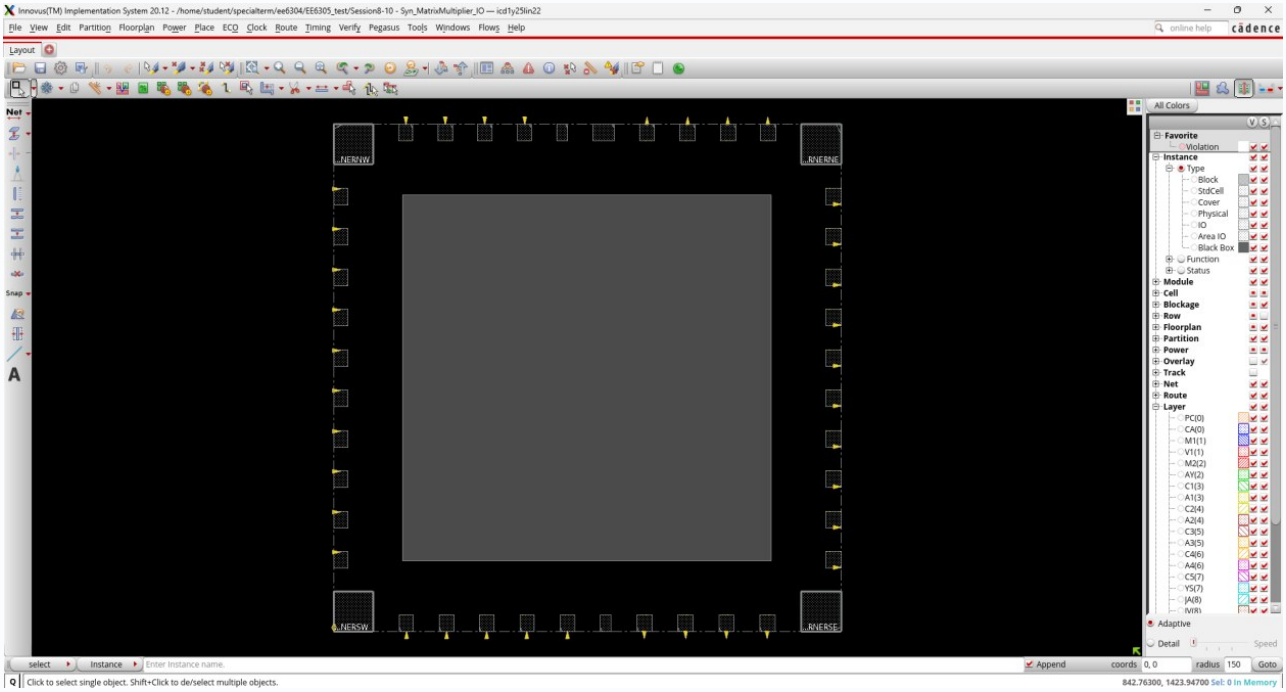
- **全局 Floorplan**：Die + Core + IO pads。
- **PG/Power Rings**：显示 vdd/gnd core ring / special routing。
- **Placement**：放大 Core，关闭遮挡的金属，看到标准单元 Rows/Cells。
- **CTS**：Clock Tree Debugger 中的 clock root、buffer、sinks。
- **Final Routing**：显示必要 Metal/Signal nets、Die/Core/IO，不要只有空白。
- **Timing**：最终 setup 和 hold summary（同一最终数据库）。
- **DRC**：最终 `No DRC violations were found` 的 Report。
- **Files**：GDSII/SDF/网表真实 `ls -lh` 和必要的 top。

E44 [实遇：助教修订，A 类] `init_design` 找不到 `AES_TOP_DP.view`

- **故障原文关键词**：`AES_TOP_DP.view`、`init_mmmc_file`。旧 `Syn_MatrixMultiplier_IO.globals` 遗留了其它项目的 MMMC view。
- **原因分支**：① `.globals` 为旧版；② 文件已换但 Innovus 仍加载旧缓存/当前目录不对；③ `.view` 确实丢失。
- **当时解决方案**：2026-10-16 助教发布修订版 `cshrc_inno` 与 `Syn_MatrixMultiplier_IO.globals`。在有授权的实验室目录复制最新版；若仅需临时验证，在 `innovus>` 输入：

```
source Syn_MatrixMultiplier_IO.globals
set init_mmmc_file Syn_MatrixMultiplier_IO.view
init_design
```

- **验收**：检查 `Syn_MatrixMultiplier_IO.view` 可读且布局显示正确 IO pad/core；不要随手编辑 tech LEF、MMMC library。



M6 官方手册：core/die/pads 与 floorplan 视图，供 init_design 恢复后核对

E45 [实遇/操作经验，B 类] csh 不认 2>/dev/null、循环、GUI 的 _ / / 难输入

- 先看 `echo $shell`。`foreach ... end` 属于 csh；`for ...; do ...; done` 属于 bash；重定向语法不应混用。
- **修复**：需要 bash 流水线时另输入 `bash`；配置软件仍按手册在 `csh` `source cshrc_inno`。VNC 键盘映射影响特殊字符时复制粘贴原命令并仔细检查结果。
- **验收**：`which innovus` 有效、`pwd` 是 Lab6 实际目录、命令输入没有缺失 `_`、`/` 和 `{ }`。

E46 [实遇但结果未完整闭环，B 类] verifyConnectivity 被系统 Killed

- `Killed` 是进程被信号中断的症状；**不能仅凭这一行断言是 OOM**，也可能是资源限额、作业系统回收等。
- **第一分支**：检查 shell `dmesg`/系统日志是否有 OOM kill（无权限则不能检查）；`free -h`、`top`、`ulimit -a`、运行日志与设计数据库是否仍可恢复。
- **处理**：保存当时 checkpoint 和日志，在可用资源下单独重试 `verifyConnectivity`；不成功就报告“未完成”，不可假写 connectivity 0。
- **历史状态**：最终报告已确认为 `verify_drc` 0，但不能据此推出上面这条因 `Killed` 终止的检查也 PASS。

E47 [实遇，A/B 类] IMPDC-495: Run RC extraction before invoking RC output

- **三分支**：① 前面的 `extractRC` 失败；② `setExtractRCMode` 保留了 postRoute/signoff 配置、触发 Quantus；③ 运行目录/corner 不对。
- 旧笔记中尝试了：

```
setExtractRCMode -reset
extractRC
rcOut -spef Syn_MatrixMultiplier_IO.spef -rc_corner typical
```

- **判定边界**：旧笔记声称该流程曾输出 preRoute estimated SPEF，但最终签核报告明确指出 Quantus **post-route signoff extraction 未完成**，老师已豁免。preRoute 估算结果**不等价于完成 post-route 寄生提取**。

E48 `[实遇，A/B 类]` `NREX-94: 11 routing layers, 1 layers are mapped to techfile`

- **场景**：postRoute medium/tQuantus RC extraction，TechFile 与布局 routing layer map 不匹配。
- **优先排查**：看 RC technology file/LEF layer map 及所选 effort，若涉及工艺提供文件，不要为消错自行重写 PDK map。
- **历史结果**：旧笔记指出 medium effort 无法走通；没有可信 postRoute RC 成功日志。

E49 `[实遇，B 类]` `IMPEXT-6192: Effort level 'low' ... not allowed`

- **场景**：postRoute low 模式缺少 captable，不能直接启用。
- **处理**：回到老师给定合法档位；若该环境只有 preRoute 估算可用，必须在证据里将它标成 preRoute，不可暗示是 signoff parasitics。

E50 `[实遇，B 类]` `IMPEXT-2900`：RC corner `cap` 不存在

- 历史 MMMC 定义中存在 `typical`，却不一定存在手册示例的 `cap`。
- **查法**：在 Innovus 检查当前 RC corners；对照 `.view` 中的定义，才决定 `rcOut -rc_corner typical`。不允许为让命令跑通虚构 corner。
- **验收**：命令选的 corner 是设计实际配置，且输出文件信息说明提取模型档位。

E51 `[实遇，B 类]` `IMPEXT-7050` / `IMPESI-2017 no coupling capacitance`

- **含义**：所需 parasitic extraction 不可用，后续时序可能基于无耦合项或降级 RC 模型继续计算；两种 warning 具体意义应看附近日志。
- **当时策略**：时序报告仍可作为 **Innovus 当前模型** 下的结果记录；不能宣称它已经包含 Quantus signoff extraction。
- **验收**：同时记录 `setExtractRCMode`、失败日志和 `timeDesign` 时间模型，避免“无违例”与“已完成先进提取”混为一谈。

E52 `[实遇/操作经验，B 类]` `restoreDesign` 指定错 Top / `.dat` 被误认为普通文件

```
restoreDesign save_design/MatrixMultiplier_signoff_v2.dat Syn_MatrixMultiplier_IO
```

- `.dat` 在该历史目录里代表保存的数据库目录；后一个参数是实际 `Syn_MatrixMultiplier_IO`，不是 `MatrixMultiplier`。
- **验收**：`pwd` 正确，DB 恢复后核实实际 Top 和 Post-route 状态；导出前仍需重新检查想要签核的指标。

E53 `[实遇/流程，B 类]` `timeDesign -postRoute` 再次触发 RC 失败

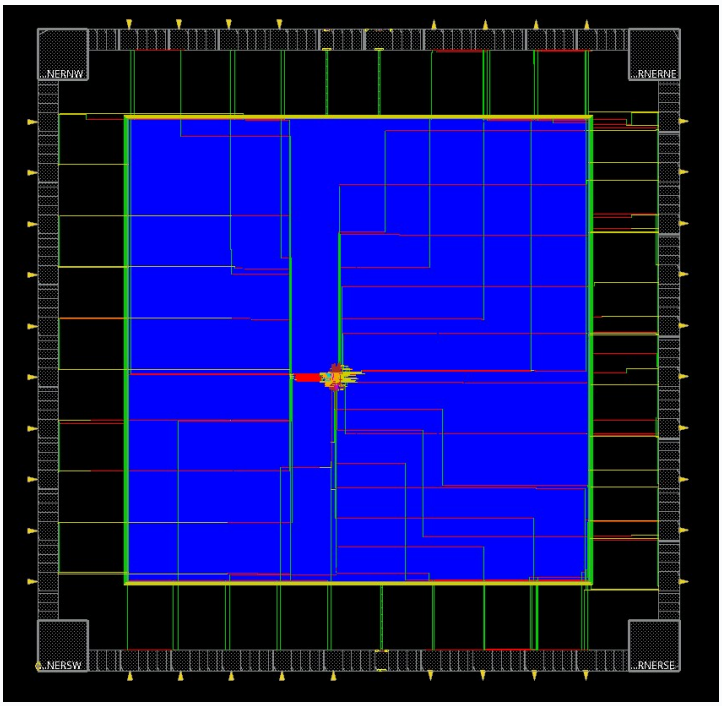
- 历史笔记提醒：设置了 postRoute extraction 后，再次调用相关 timing flow 可能触发 Quantus 或 tQuantus；此时失败来自 extraction 配置，不等于 RTL 功能错。
- **策略**：先查当前 RC extraction mode、脚本是否重设 mode；必要时回退到老师允许的模型并**明确标注模型局限**。
- **禁止**：为获得漂亮 WNS，静默切换 RC/corner 然后与旧 post-route signoff 报告直接比较。

E54 `[手册/预防，C 类]` `source scripts/CHIP.tcl` 一键流程脚本跑到中途失败

- 用 `grep -nE 'init_design|sroute|addFiller|timeDesign|rcOut|streamOut' scripts/CHIP.tcl` 找失败阶段；保留 console/log。
- **原因分支**：① 老 `.globals`；② 老 `MET1(1)` layer 标识；③ 当前文件路径/版本不匹配；④ 脚本中调用 GUI-only 的 `ctd_win`，但使用 `-no_gui`；⑤ RC 提取触发兼容性问题。
- **处理**：保留前面 checkpoint，定位最早失败的 Tcl，再从可恢复阶段继续；不可假设一键脚本 0 exit=物理验证全清。

E55 [实遇与最终报告，A 类] Lab 6 最终状态不要跨时间拼接

- Hold WNS：Post-CTS **-2.492 ns / 53** → Hold Opt **-0.001 ns / 5** → DRV Opt **-0.014 ns / 21** → Final routed **+0.419 ns / 0**。
- DRC：post-filler **19** → filler fix **1** → ECO route **0**。
- 最终 Setup WNS **+87.329 ns**、Hold WNS **+0.419 ns**，TNS/violating paths 0；Innovus `verify_drc` 0。
- **证据限制**：这是教学实验层面的 Innovus clean；不是 full foundry DRC/LVS、独立 parasitic signoff、多 PVT corner 签核。



M6 官方手册：Routing 后的 layout 示意；**最终 0 DRC 的证据仍须看本轮 verify_drc 报告**

E56 [预防，C 类] 后端先修 DRC 还是先导出 GDS?

- 原则上采用本次最终有效数据库：布线 / filler → DRC + routing ECO → 最终 timing / DRV → 再导出当前 GDS/SDF/netlist。
- 如果生成 GDS 后又改了 route，旧 GDS 不再代表新数据库，必须重新导出；通过 `ls -lh` /文件时间戳核对。
- 只关闭图层、删掉 marker、把 DRC report 截图裁掉，都不能修复数据库。

专题 A 补充｜考试实测错误链

9. 额外：Mock1 实测排错与考试迁移

> Mock1 Secret Code 是**用来复现 Lab 3–5 前端流程**，不等于 Lab 3 官方 FullAdder。考试必须依题目规定，而不是按 Mock1 的文件名或 10 ns 演示时钟套用。

问题	当时明确处理方式
`script_path` 未定义	在 `design_vision>` 先 `cd` 正确 Synthesis，再 `set script_path [file normalize ./scripts]`
`file exists=0`	打印 `pwd`、`puts $script_path` 和 `file exists`，查是否误入 Behavioral
`UID-4`	按 `read_verilog → current_design → link` 建立有效 Current Design
旧 Tcl Top `secretcode`	修为真实 `secret_code_four`，并同时检查两份 RTL source 文件
`Schematic.1/2` 只有层次框	检查 compile 与子层次 M1；不要找不到 NMOS 就认为综合失败
找不到 `.sdc`	查 `write_sdc` 和 Design Vision cwd，必要时在 compile 后显式导出
`-max_paths 5` 还是 `10`	两者只是**报告展示最多多少条路径**，不影响综合；用 5 即可，真正要读 slack/path
要做 Single/Four 两个版本	分别设 Top、重跑并单独命名网表和报告，避免覆盖

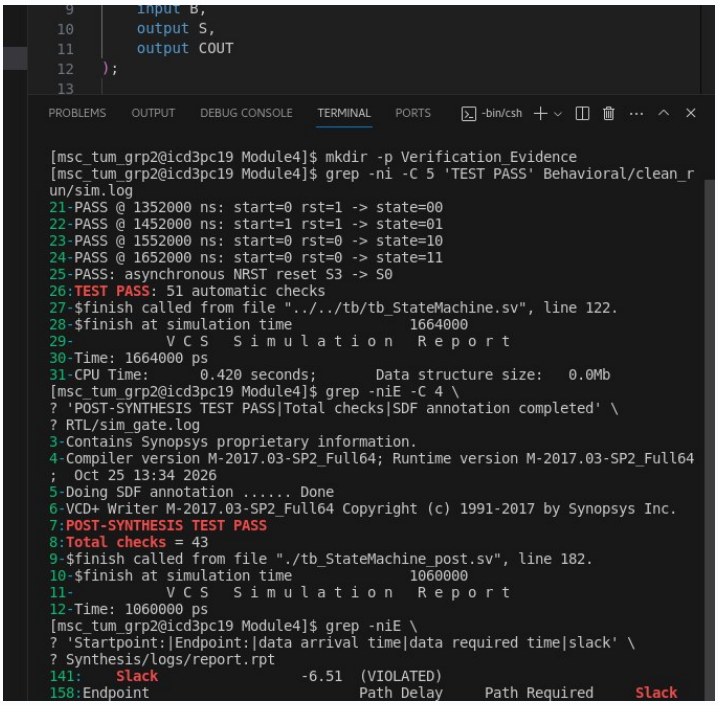
重要：`report_timing` 的 `data arrival time` 是这个 report 路径在当前时序模型下的延迟组成；`slack` 是 required−arrival；SDF 波形里量输入/输出跳变是另一个概念。**别把 “+99ns slack” 当成 “99ns propagation delay”**。

E41 `[实遇：Mock2，B 类]` VCS 编译 `.sv` 缺少 `-sverilog`，`logic clk` 报语法错误

- **可恢复历史现象**：源文件用 `logic clk = 1'b0;`，但 VCS 命令按 Verilog 模式解析，导致 SV 关键字/声明附近 syntax error。
- **先查**：`grep -nE 'logic|always_ff|always_comb|bit|int' tb*.sv`；阅读 VCS compile command 中是否启用 SV。
- **修复示例**：

```
vcs -full64 -sverilog -debug_access+all   ./tb_serial_adder.sv ./serial_adder.v   -o ./simv -l compile.log
./simv -l run.log
```

- **当时反馈**：补充 `-sverilog` 后，之前一轮行为仿真曾打印 `PASS ALL`；**这不能替代本轮门级/SDF 和 Power 的独立确认**。



用户 M4 证据包：VCS 自检终端截图（示例：应看到真实 Checker PASS，而非只有 $display）

E42 `[实遇：Mock2，B 类]` `UID-3` → `LBR-1` / `LINK-5`：`your_library.db` 是占位符

错误原文（可搜索）：

```
Warning: Can't read link_library file 'your_library.db'. (UID-3)
Information: Building the design 'HDBSLT20_ADDH_1'. (HDL-193)
Warning: Cannot find the design 'HDBSLT20_ADDH_1' in the library 'WORK'. (LBR-1)
Warning: Cannot find the design 'HDBSLT20_AO32_1' in the library 'WORK'. (LBR-1)
Unable to resolve reference ... (LINK-5)
24 unresolved references
```

- **根因优先级**：① `link_library` 里写着虚构的 `your_library.db`；② 正确 GF22 `.db` 不在 `search_path`；③ 顶层网表/库属于不同的 PDK/版本；④ 读入的是 mapped gate netlist 却以 RTL 库方式 link。
- **修复动作**：先在 Linux 只读确认真实 `.db` 路径；在 `dc_shell>` 输出 `printvar search_path`、`printvar target_library`、`printvar link_library`；使用学校提供的 `common_setup_gf22.tcl` 设置**真实**库文件，然后重新 `link; check_design`。
- **不同于 VCS**：`HDBSLT20_*` 在 DC 中需要 `.db` link，在 VCS 门级模拟则需要对应的标准单元 **Verilog** simulation model（例如 `library.f` 引入）。
- **状态**：历史对话明确诊断根因，但没有恢复到“24 unresolved 全清零”的最终 console PASS；不得写成已验证修复。

E43 `[实遇：Mock2，B 类]` `PWR-362`: No switching activity has been annotated

原始上下文：`read_saif ... -instance tb_serial_adder/U1` 后出现 `No switching activity has been annotated (PWR-362)`。

- **首先排除 E42**：若综合网表 GF22 cell references 没 link 成功，不能把活动映射异常仅归因于 instance。
- **接着检查 TB 层次**：先前 Mock2 的真实 DUT 实例可能是 `tb_serial_adder/dut`，不是 `tb_serial_adder/U1`，需以最新 TB、SAIF 中的 `INSTANCE` 树和报告为准。
- **修复顺序**：`link` 0 unresolved → `read_saif` 用真实 DUT hierarchy → `report_saif -hierarchy` 和 `-missing` → `report_power`。若仍 0%，检查是否从正确的门级仿真生成活动与时间窗。

- **验收**：nets/pins/ports 得到实际 user-annotated switching activity；仅消失 `PWR-362` 不代表 Power estimate 条件完全一致。

E57 `[预防：考试迁移，C 类]` RTL、综合网表、SDF 彼此不同版本

- **现象**：行为 PASS、门级 FAIL；或 `$sdf_annotate` 实例找不到；综合单元名称与库不一致。
- **诊断顺序**：`sha256sum` 检查 RTL 与综合输入 → 记录综合输出时间 → 检查 `Syn_TOP.v` module 名 → 核对 SDF `DESIGN` / `INSTANCE` → TB DUT 的实例名、端口。
- **验收**：4 阶段全部对准同一组提交，避免 “前一晚生成的 SDF + 今天的 gate netlist”。

E58 `[预防：考试迁移，C 类]` `readmemh` 输入数据格式被误读

- **例子**：`$readmemh` 把 `010` 当十六进制 `0x10`，不是二进制的 `2`；若向量是 binary bit strings，应该使用 `$readmemb`。
- **验收**：TB 在读入后显示首尾测试向量并与题目向量格式核对；通过 refmodel 算期望值而非机械复读 DUT 输出。

10. 提交报告前 12 项签核（任何题目通用）

- [ ] 题目 Spec 的 Top、端口方向、位宽、signedness、Reset 极性、Clock、Latency/Throughput 已明确。
- [ ] 当前 RTL 与 Synthesis/sources 一致；关键文件非零大小、版本正确。
- [ ] T1 VCS 重新编译，Testbench 自检真的 PASS（不只是能开 DVE）。
- [ ] T2 Design Vision 目标 GF22 库正确，真正 `compile`；mapped cells/area/timing 可信。
- [ ] T2 constraints 实际生效；违例按类型区分，未约束 IO 不假装已 STA signoff。
- [ ] `Syn_TOP.v` / SDF / report.rpt 为当前设计本轮生成；若任务要求 SDC 则确实存在。
- [ ] T3 VCS 包含 `-f library.f`、DUT 为**综合网表**；无 unresolved module。
- [ ] T3 `$sdf_annotate` 指向真实 DUT 实例，警告已检查；门级自检 PASS。
- [ ] T3 当前运行产生了正确、非空 VPD，时间戳不是旧文件。
- [ ] T4 VPD→VCD→SAIF 步步成功，`read_saif` instance 层次正确、annotation 有效。
- [ ] T4 report_power 写明 corner/clock/stimulus/coverage；比较条件一致。
- [ ] 若题目要求 Lab 6：最终 routed DB、setup/hold/DRV/DRC、GDS/SDF/netlist 是同一状态；SPEF 豁免仅限老师本轮明确要求。

10.1 错误已修复的判断法

修复 ≠ 报错消失。必须有四层证据：① 根因对应；② 当前步骤无致命 error；③ 本轮输出新鲜且内容正确；④ 下游检查 PASS。例如改了 `script_path` 只能证明能打开 Tcl，**不能直接宣称 GF22 synthesis PASS**。

10.2 不要做的危险操作

- 不为了消 Warning 改 PDK/standard-cell Verilog 或随意关闭 timing checks。
- 不把 `/home/msc_tum_grp1/...` 硬编码照搬到自己的 `grp2` 账户。
- 不因为 Terminal 的 cwd 正确就认为**已启动的** Design Vision cwd 也正确。
- 不修改老师设定的工艺 corner、clock uncertainty、输出有效周期来“制造好看数据”。
- 不在公开/私有 GitHub 上传 GF22 PDK、cell library、EDA 安装包、学校许可或受限制的实验文件；GitHub 工作流仅同步获准共享的 RTL、TB、自己的 Tcl/说明文件。
- 不把自动脚本退出码为 0、图层隐藏、单张截图、报告数字存在当作完整签核。

11. 最常用终端命令速记

```
# 观察环境/路径
pwd
ls -lah
which vcs
which design_vision
which innovus

# 查真实文件位置和版本
find ~/NM6008 -name 'library.f' -o -name 'Syn_*.sdf'
find . -name '*.vpd' -print
sha256sum Behavioral/Top.v Synthesis/sources/Top.v

# 找日志第一条关键错误
grep -niE 'Error|Fatal|Warning|SDF|FAIL|PWR-|VIOLATED' compile.log sim.log | head -70

# 看 SDC 和综合关键命令
grep -RniE 'create_clock|read_verilog|current_design|compile|write_sdf|write_sdc' scripts/

# 查看压缩 timing summaries（若为 .gz）
zgrep -iE 'WNS|TNS|Violating|Negative Slack' timingReports/*.summary.gz 2>/dev/null | head -50

# 使用 GitHub 的获准共享源码（先 cd 到已 clone 的 repo）
git status --short
git pull --ff-only origin main
```

注意 `find ... -o ...` 为演示性查找，正式组合多个条件需加括号避免优先级误判；`sha256sum` 例子也要替换成当前真实路径与文件名。**如果没有目标文件，就停下来定位目录，不要假装运行命令成功。**

---

12. 来源索引：出现问题时对应看哪一页

来源	查阅页/节	对应内容
`NM6008 Module 3 - Full-Adder`（2026）	P4–9，§2.1	Behavioral RTL / TB / VCS / DVE
同上	P10–16，§2.2	GF22 setup、Tcl、Design Vision、netlist/SDF/SDC、report
同上	P16–18，§2.3	`rtl/`、`library.f`、SDF TB 反标、门级 VCS
同上	P19–20，§2.4	PowerAnalysis、VPD/VCD/SAIF、Power Compiler 四报告
`NM6008 Module 4 - Verilog State Machine`（2026）	P4–7、P8–9	FSM/NRST、100ns clock、综合/功耗复用 M3、glitch optional
`NM6008 Module 5 - Verilog Matrix Multiplier`（2026）	P3–5	signed 8×8、17-bit OUT、OUT_STROBE、Datapath/FSM、复用 M3 流程
`NM6008 Module 6 - Layout of Matrix Multiplier`（2026）	P3–21，§2–3	Innovus 导入、PG、Floorplan、IO filler、Placement、CTS、Route、DRC、导出
`NM6008_Frontend_Manual_Mock1_SOP_Updated.pdf`	“T2 INCIDENT REPLAY” 及故障救援章节	Mock1 的 cwd/`script_path`/`UID-4`/Top/schematic/STA 实机补丁
`NM6008_Module4_Report.pdf`	§3–7	M4 FF/面积/Slack、SDF、SAIF、Optional 对照结果

来源	查阅页/节	对应内容
`NM6008_Module5_Report_Final(2).docx`	§3–8	M5 18/18、54 SDF warnings、buffer/area、SAIF 覆盖与 100 MHz 活动
`NM6008_Module6_Report_Final(2).docx`	§3–7	M6 2.5ns hold uncertainty、19→1→0 DRC、教师 layer 修订、Quantus/GLIBC 豁免、最终 WNS
`NM6008_Modules3-5_陌生Lab四阶段手把手SOP_原详细版.md`	A/B/C/D/G	四阶段流程、详细命令模板、通用故障原因、日志验收

特别提醒：老师的 M3 原文可能使用 `synosys_linux_...` 拼写、M6 使用旧 layer 名、M3 同时出现 `RTL` / `rtl`。优先按真实磁盘文件名与老师最新通知执行；不要以 PDF 中的每个字符都绝对正确为前提。

---

13. 考试故障决策树（最后 1 页）

```
启动工具失败?
  ├─ Command not found → 看 E01、§1（csh / source / which）
  └─ 有工具但 Tcl 报错 → 先认提示符，检查 E09–E12（cwd / script_path / Current Design / Top）

VCS 失败?
  ├─ 语法错误 → E02–E04
  ├─ 数据库版本冲突 → E05
  ├─ 没有波形 → E06–E07
  └─ 实际 TB 检查失败 → E08 / F01–F05

综合结果不对?
  ├─ 没生成 netlist/SDF/SDC → E13 / E17
  ├─ 只见层次框/看不到 MOS → E14
  ├─ 时钟/约束/slack 搞不懂 → E15–E16 / F03
  └─ GF22 库/引用失败 → E18

门级不对?
  ├─ unknown cell/library.f → E19 / E24
  ├─ SDF warning/没注释 → E20–E21
  ├─ X/glitch/timing checks → E22
  └─ 没有 VPD → E23

功耗不对?
  ├─ VPD→VCD→SAIF 某个文件不存在 → E25–E26
  ├─ report_saif 0% / 默认活动 → E27
  ├─ unmatched source objects → E28
  ├─ report_saif 命令不支持 → E29
  └─ 输出文件/功耗比较异常 → E30 / F07

Innovus 不对?
  ├─ Import warnings → E31
  ├─ SRoute layer identifier 错 → E32
  ├─ 看不到 stdcell / metal 遮挡 → E33
  ├─ hold violation → E34
  ├─ DRC M1 short / filler → E35
  ├─ Quantus GLIBC / SPEF → E36
  ├─ utilization / density 误读 → E37
  └─ 导出 / timing / 截图签核 → E38–E40
```

操作底线：每次遇见新报错，先把原始错误关键词、失败阶段、`pwd`、相关脚本/命令、修复结果和是否成功验收补进本手册。这样它才会随着实际 Lab 累积成为可靠 SOP，而不是不可验证的“万能经验大全”。

---

14. 可靠性审计与覆盖声明（考试使用前的必要限制）

- **本指南不是承诺全部历史消息均可检索**：部分旧对话中的内嵌截图无法从历史文件库恢复为独立像素文件。原图不可恢复时，正文记录可检索的原始 error text；用 M4 证据包中的实验图、M3/M6 手册中的 GUI 图作为阶段性视觉参照，并在图下注明来源。
- **历史操作笔记与最终报告冲突时**：以最后一次可核对报告界定完成度。Lab6 早期笔记的 preRoute SPEF 并不能推翻最终报告“Quantus post-route extraction 失败、教师豁免”。
- **未完成情况仍保留**：Mock2 Serial Adder 的 `UID-3/LBR-1/LINK-5` 及 `PWR-362` 有明确诊断，但没有被当前资料证明本轮修复完全 PASS；M6 `verifyConnectivity` 曾被 `Killed`，不能假写 connectivity clean。
- **不同来源分类**：`[实遇]` 可以是报错确实出现；它与“修复成功”是不同的断言。看到具体命令仍必须替换 Top、真实库路径、TB、DUT 实例、Clock、工具版本。
- **覆盖维度**：Behavioral RTL/TB → GF22 DC 综合/STA → SDF 门级仿真 → SAIF Power → Innovus 导入/PG/Place/CTS/Route/DRC/RC/Export；Mock1/2 纳入迁移案例。真实世界中尚未遇到的新错误不可能被静态指南穷尽。

历史依据（不是外部通用网页）

1. 《NM6008 Module 3 - Full-Adder》（2026，22 页），P4–20。
2. 《NM6008 Module 4 - Verilog State Machine》（2026，9 页），P3–9。
3. 《NM6008 Module 5 - Verilog Matrix Multiplier》（2026，5 页），P3–5。
4. 《NM6008 Module 6 - Layout of Matrix Multiplier》（2026，22 页），P3–22。
5. `NM6008_Frontend_Manual_Mock1_SOP_Updated.pdf`，实机事件回放与操作增补。
6. `NM6008_Module4_Final_Report_Package.zip`，用户报告和可提取的实验原始截图。
7. `NM6008_Module5_Report_Final(2).docx`、`NM6008_Module6_Report_Final(2).docx`，最终技术数据及完成边界。
8. 历史 `M6_Innovus布局布线全流程.md`（较早操作笔记；部分提取成功表述与最终报告冲突）。
9. 2026-10-28 Secret Code / Serial Adder 的可检索对话片段与报错原文。

> **考场纪律**：先保存第一条 Error 和完整 context，不要把未验证的命令试验结果写进最终报告。任何“修好”都必须具备重新运行后的有效产物、日志和下游测试结果。